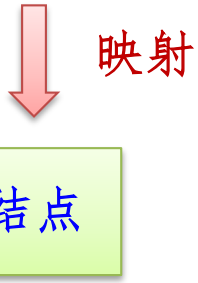
2.3 线性表的链式存储结构

2.3.1 线性表的链式存储结构—链表

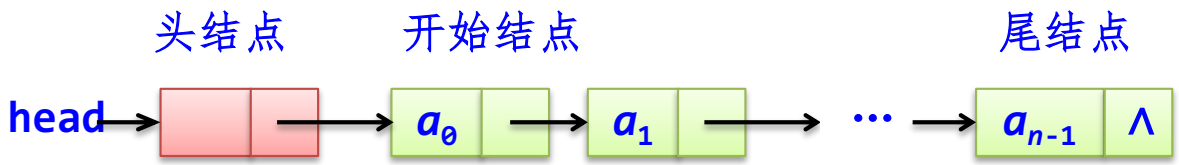
线性表：

$$(a_0, a_1, \cdots, a_i, a_{i+1}, \cdots, a_{n-1})$$



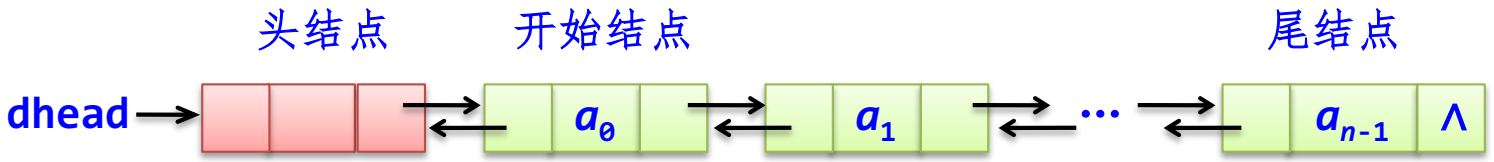
结点包含有元素值和前后继结点的地址信息

- 如果每个结点只设置一个指向其后继结点的指针成员，这样的链表称为线性单向链接表，简称**单链表**。



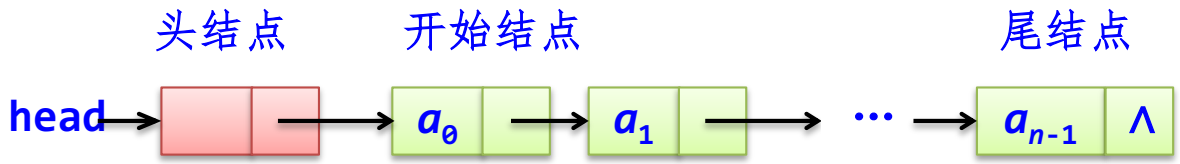
（a）单链表

- 如果每个结点中设置两个指针成员，分别用以指向其前驱结点和后继结点，这样的链表称之为线性双向链接表，简称**双链表**。



（b）双链表

2.3.2 单 链 表



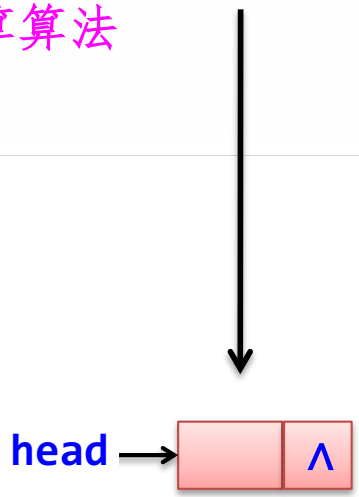
每个结点为LinkNode类型，包括存储元素的数据属性data和存储后继结点的指针next。

```
template <typename T>
struct LinkNode                                  //单链表结点类型
{   T data;                                      //存放数据元素
    LinkNode<T>* next;                           //下一个结点的指针

    LinkNode():next(NULL) {}                     //构造函数
    LinkNode(T d):data(d),next(NULL) {}          //重载构造函数
};
```

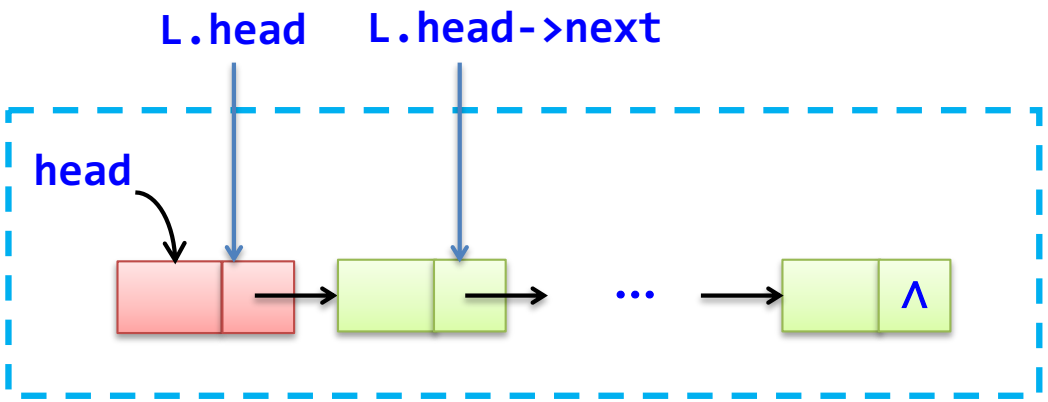
单链表类模板LinkList

```
template <typename T>
class LinkList                          //单链表类模板
{
public:
   LinkNode<T>* head;                   //单链表头结点
//基本运算算法
};
```



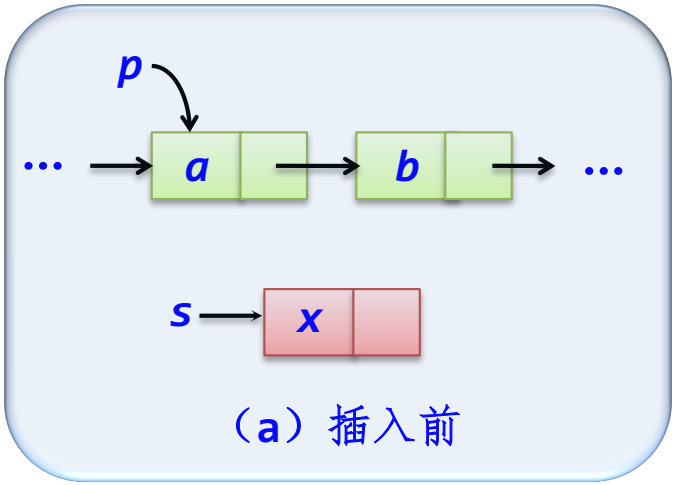
结点引用方式：

单链表对象L：

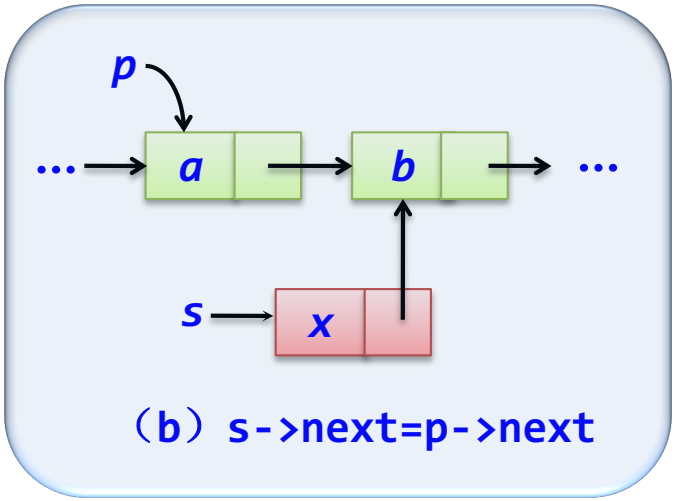


1．插入和删除结点操作

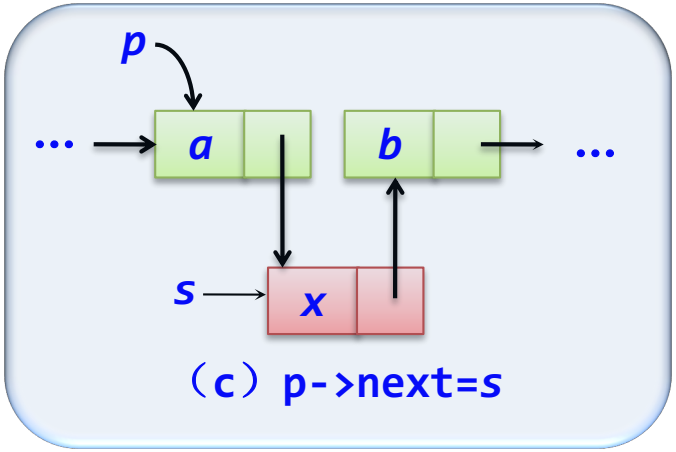
插入结点操作：将结点s插入到单链表中p结点的后面。



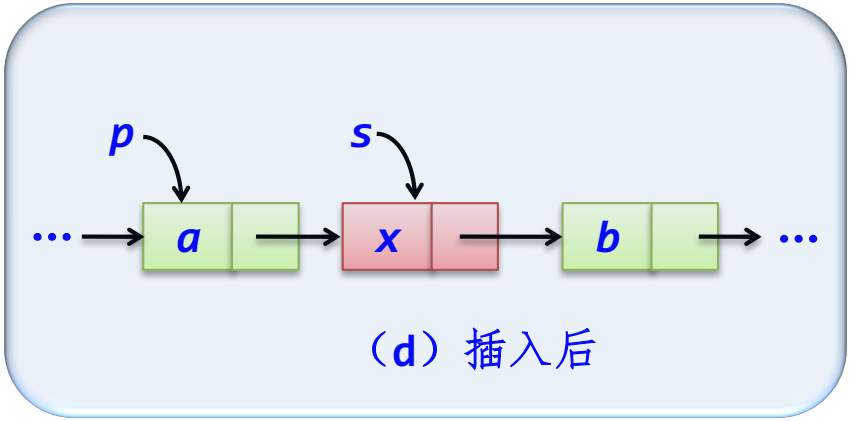
（a）插入前



（b）s->next=p->next



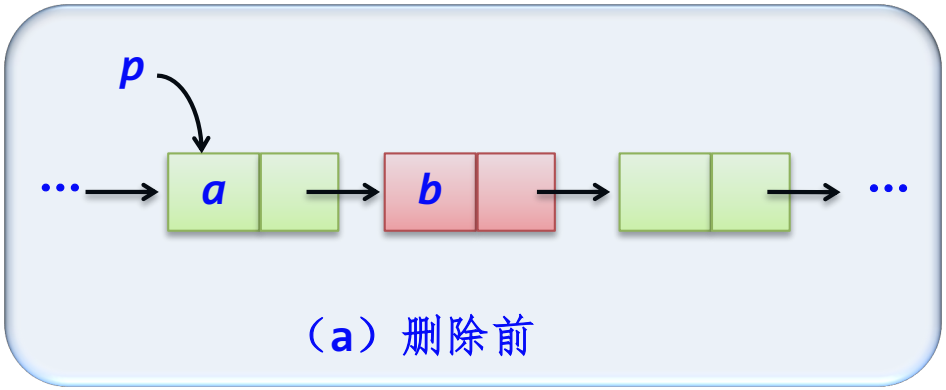
（c）p->next=s

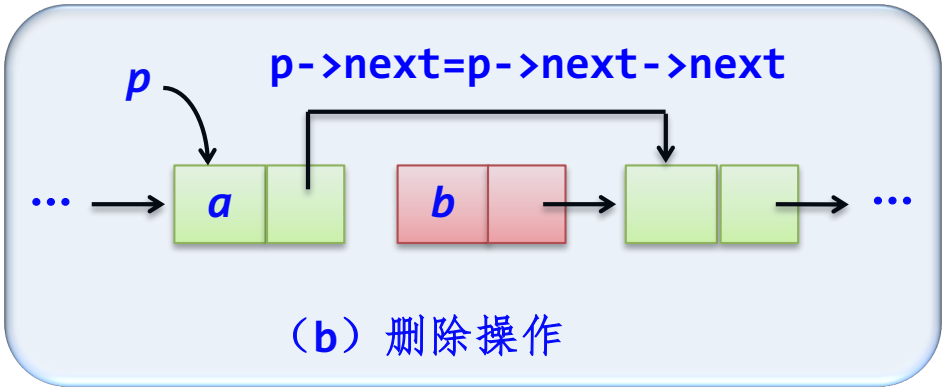


（d）插入后

```
s->next=p->next;
p->next=s;
```

删除结点操作：删除单链表中*p*结点的后继结点。





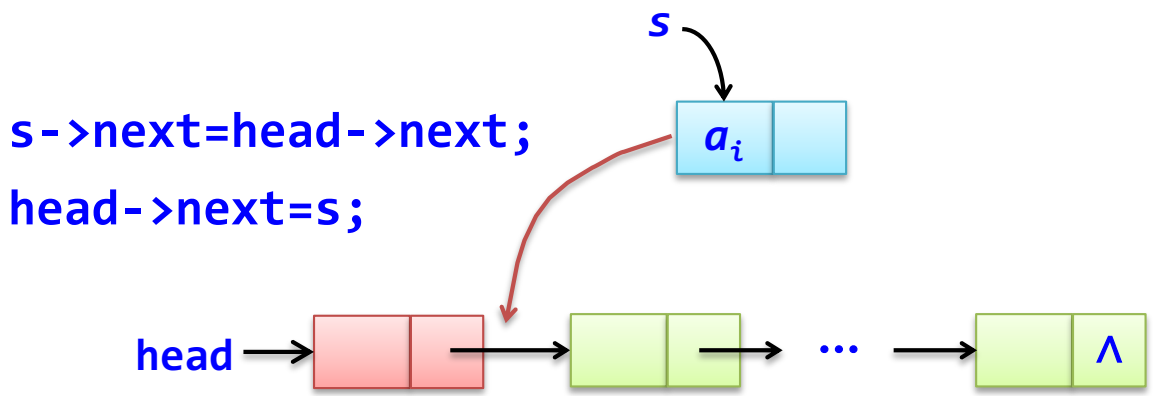

```
q=p->next;              //q指向被删结点
p->next=q->next;        //从单链表中删除结点q
delete q;               //释放空间
```

2．整体建立单链表

- 通过一个含有*n*个元素的*a*数组来建立单链表。
- 建立单链表的常用方法有两种：头插法和尾插法。

头插法建表

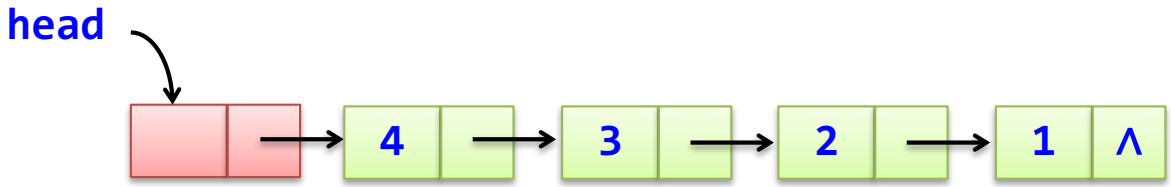
- 从一个空表开始，依次读取数组a中的元素。
- 生成新结点s，将读取的数据存放到新结点的数据成员中。
- 将新结点s插入到当前链表的表头上。



```
void CreateListF(T a[],int n)                          //头插法建立单链表
{  for (int i=0;i<n;i++)                               //循环建立数据结点
   {  LinkNode<T>*  s=new LinkNode<T>(a[i]);//创建数据结点s
      s->next=head->next;                              //将结点s插入head结点之后
      head->next=s;
   }
}
```



a=[1,2,3,4]，调用CreateListF(a,4)

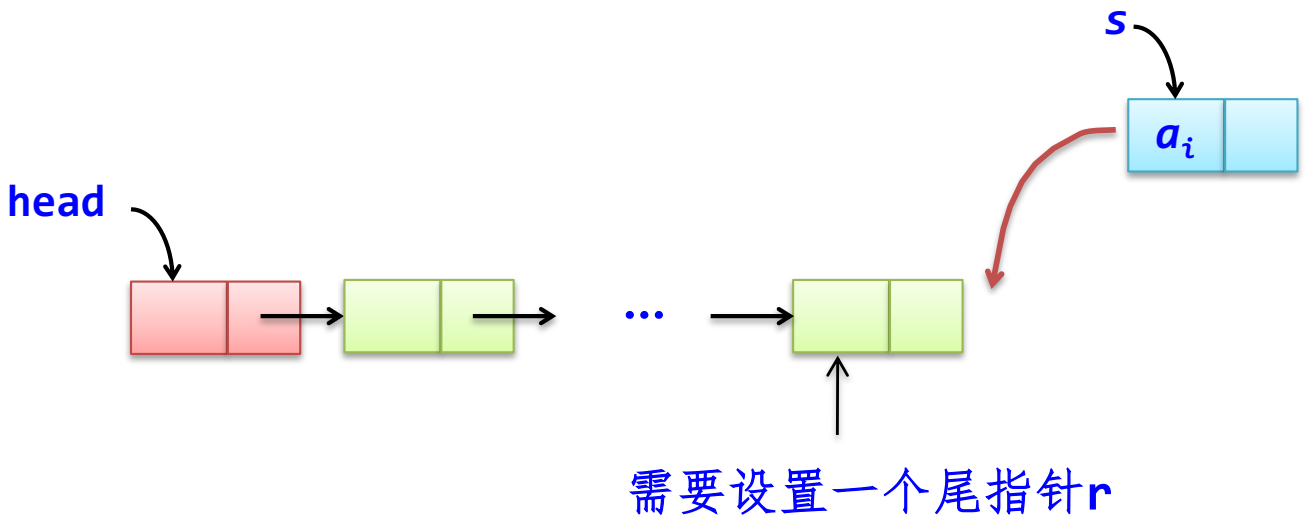




头插法建立的单链表中数据结点的次序与*a*数组中的次序正好相反

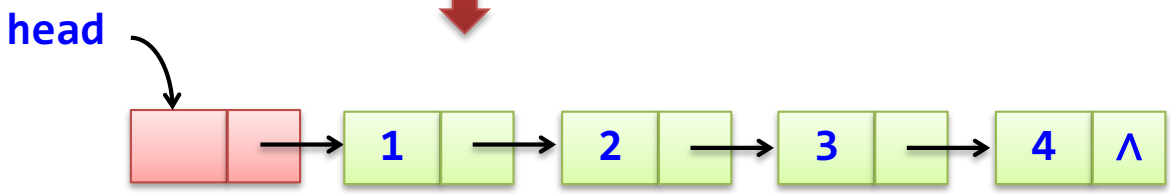
尾插法建表

- 从一个空表开始，依次读取数组**a**中的元素。
- 生成新结点**s**，将读取的数据存放到新结点的数据成员中。
- 将新结点**s**插入到当前链表的表尾上。



```
void CreateListR(T a[],int n)          //尾插法建立单链表
{  LinkNode<T>* s,*r;
   r=head;                             //r指向尾结点,开始时指向头结点
   for (int i=0;i<n;i++)               //循环建立数据结点
   {  s=new LinkNode<T>(a[i]);         //创建数据结点s
      r->next=s;                       //将结点s插入结点r之后
      r=s;
   }
   r->next=NULL;                       //将尾结点的next域置为NULL
}
```

a=[1,2,3,4]，调用CreateListR(a,4)



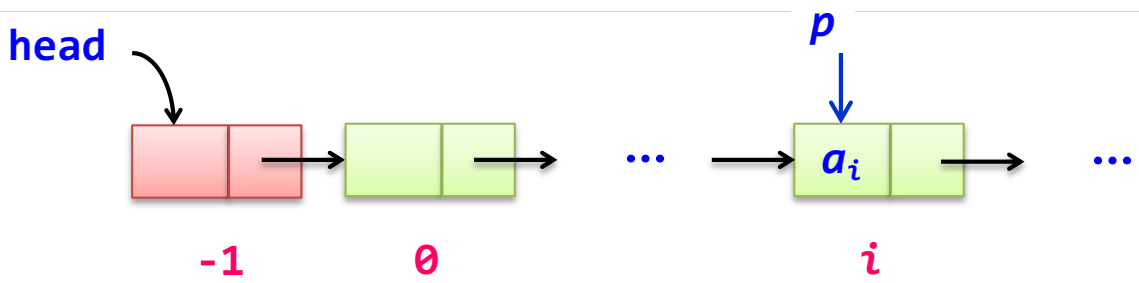


尾插法建立的单链表中数据结点的次序与a数组中的次序正好相同

3．线性表基本运算在单链表中的实现

查找序号为i（-1≤i≤n-1，n为单链表中数据结点个数）的结点

```
//*********************************************
//序号i的正确范围：-1≤i<n，超出范围返回NULL
//i=-1时返回头结点head
//i≥0并且i<n时返回序号i的结点
//*********************************************
LinkNode<T>* geti(int i)                    //返回序号i的结点
{  if (i<-1) return NULL;                   //i<-1返回NULL
   LinkNode<T>* p=head;                     //首先p指向头结点
   int j=-1;                                //j置为-1(可以认为头结点序号为-1)
   while (j<i && p!=NULL)                   //指针p移动i+1个结点
   {  j++;
      p=p->next;
   }
   return p;                                //返回p
}
```



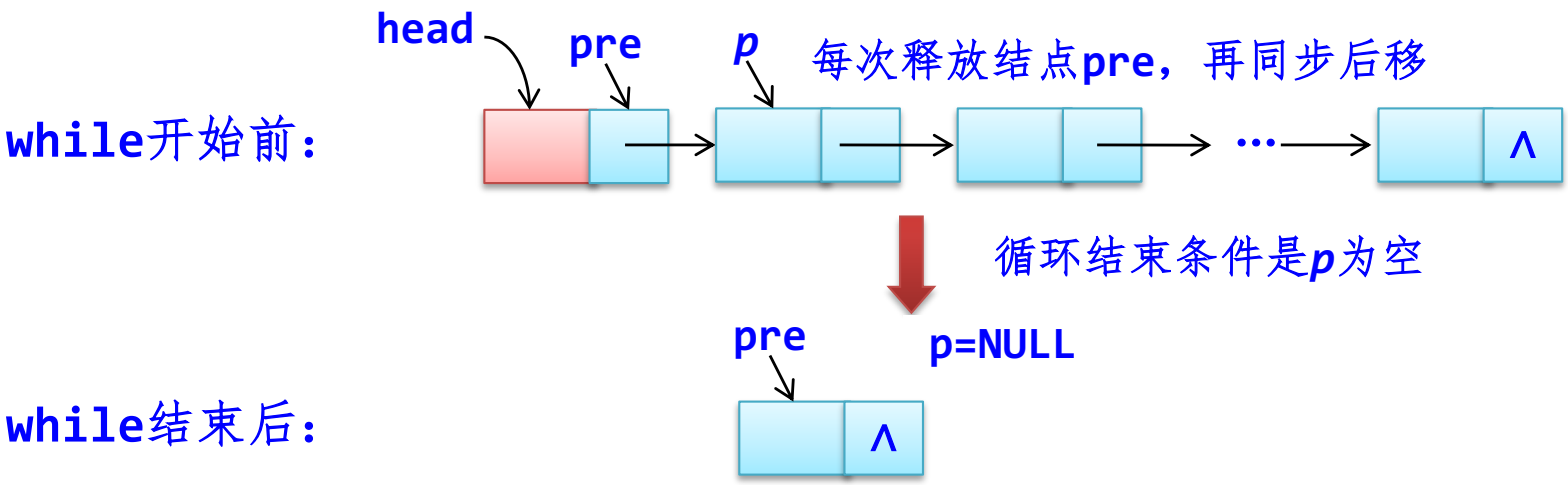
（1）单链表的初始化和销毁

构造函数

```
LinkList()                    //构造函数,创建一个空单链表
{
   head=new LinkNode<T>();
}
```

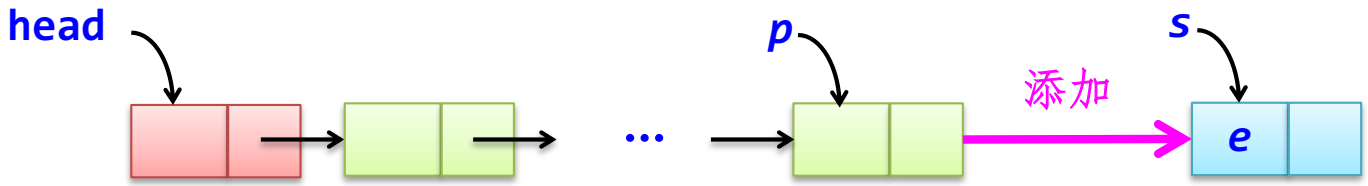
析构函数

```
~LinkList()                          //析构函数,销毁单链表
{   LinkNode<T>* pre,*p;
    pre=head;p=pre->next;
    while (p!=NULL)                  //用p遍历结点并释放其前驱结点
    {   delete pre;                  //释放pre结点
        pre=p; p=p->next;            //pre,p同步后移一个结点
    }
    delete pre;                      //p为空时pre指向尾结点,此时释放尾结
点
}
```



（2）将元素e添加的线性表末尾Add(e)

```
void Add(T e)                          //在单链表末尾添加一个值为e的结点
{  LinkNode<T>* s=new LinkNode<T>(e);        //新建结点s
   LinkNode<T>* p=head;
   while (p->next!=NULL)                     //查找尾结点p
      p=p->next;
   p->next=s;                                //在尾结点之后插入结点s
}
```



（3）求线性表的长度Getlength()

```
int Getlength()                          //求单链表中数据结点个数
{  LinkNode<T>* p=head;
   int cnt=0;
   while (p->next!=NULL)                 //找到尾结点为止
   {  cnt++;
      p=p->next;
   }
   return cnt;
}
```



若像顺序表中一样，在单链表中设置一个长度length，插入时length++，删除时length--。那么求长度的时间复杂度为O(1)了。

（4）求线性表中序号为i的元素GetElem(i,&e)

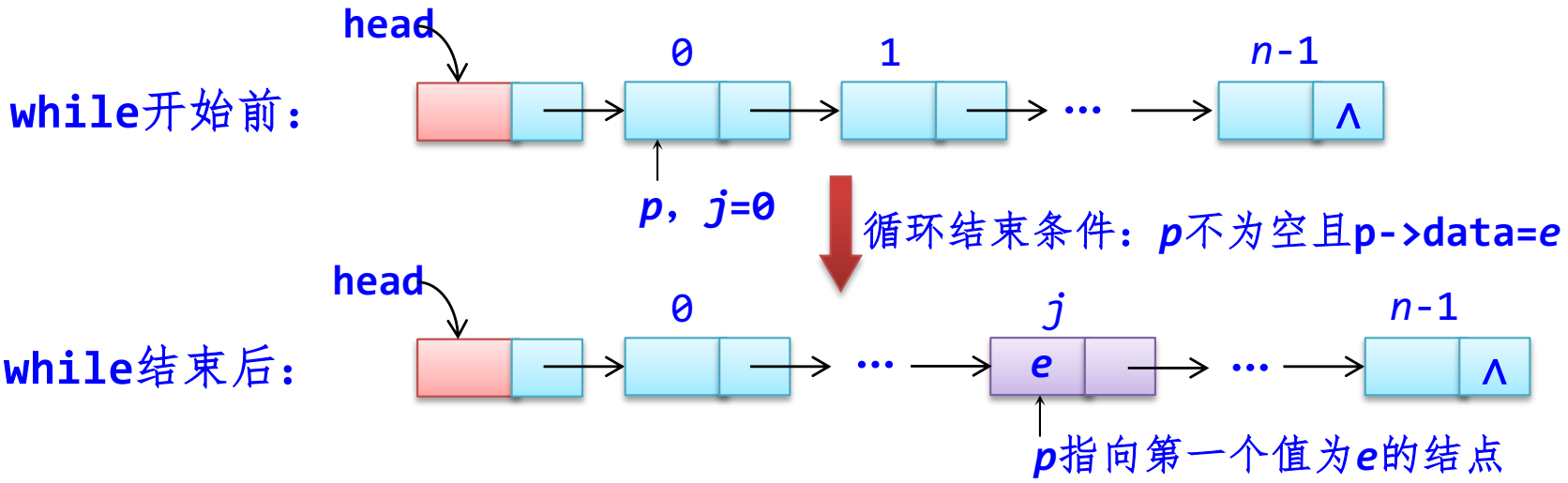
```
bool GetElem(int i,T& e)                    //求单链表中序号i的结点值
{  if (i<0) return false;                   //参数i错误返回false
   LinkNode<T>* p=geti(i);                  //查找序号i的结点
   if (p!=NULL)                             //找到了序号i的结点p
   {  e=p->data;
      return true;                          //成功找到返回true
   }
   else                                     //不存在序号i的结点
      return false;                         //参数i错误返回false
}
```

（5）设置线性表中序号为i的元素SetElem(i, e)

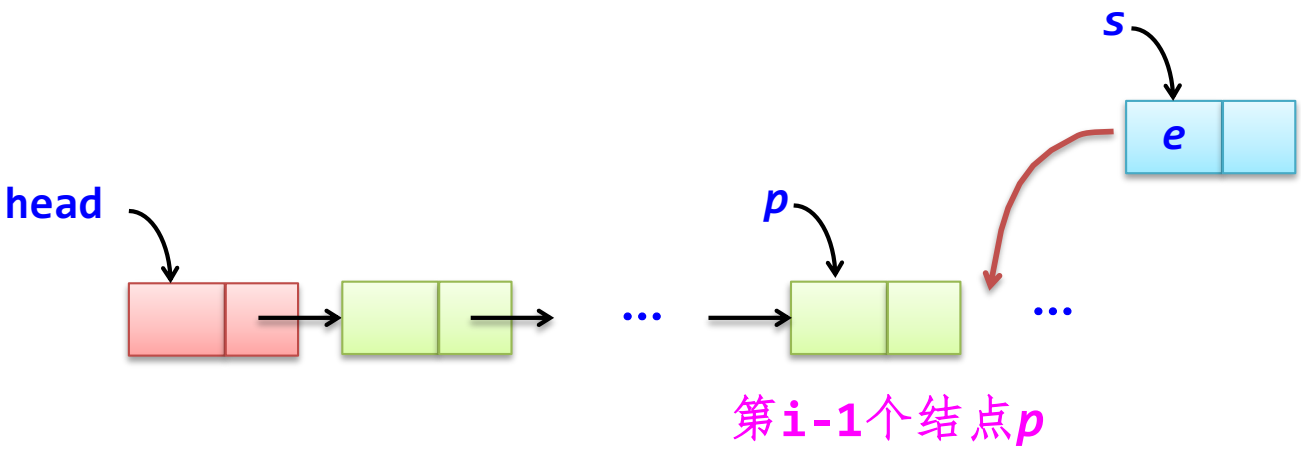
```
bool SetElem(int i,T e)                    //设置序号i的结点值
{  if (i<0) return false;                  //参数i错误返回false
   LinkNode<T>* p=geti(i);                 //查找序号i的结点
   if (p!=NULL)                            //找到了序号i的结点p
   {  p->data=e;
      return true;
   }
   else                                    //不存在序号i的结点
      return false;                        //参数i错误返回false
}
```

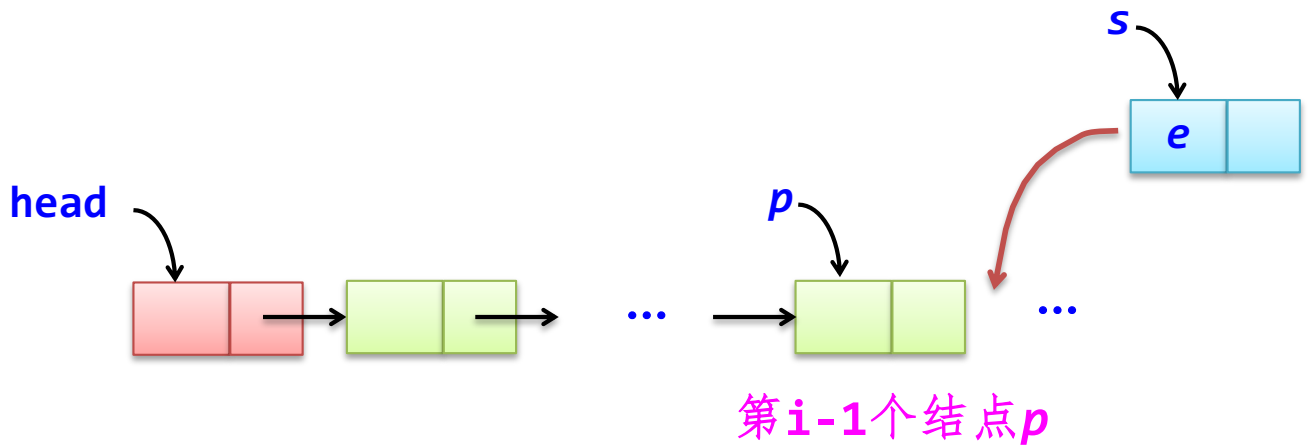
（6）求线性表中第一个值为e的元素的逻辑序号GetNo(e)

```
int GetNo(T e)                          //查找第一个为e的元素的序号
{  int j=0;                             //j置为0，p指向首结点
   LinkNode<T>* p=head->next;
   while (p!=NULL && p->data!=e)        //查找第一个值为e的结点p
   {  j++;
      p=p->next;
   }
   if (p==NULL) return -1;              //未找到时返回-1
   else return j;                       //找到后返回其序号
}
```



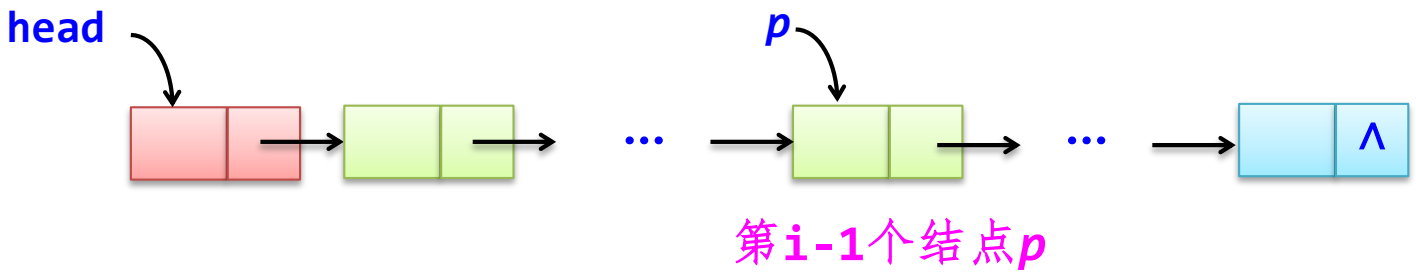
（7）在线性表中插入e作为第i个元素Insert(i，e)






```
bool Insert(int i,T e)                    //在序号i位置插入值为e的结点
{  if (i<0) return false;                 //参数i错误返回false
   LinkNode<T>* s=new LinkNode<T>(e);//建立新结点s
   LinkNode<T>* p=geti(i-1);              //查找序号i-1的结点
   if (p!=NULL)                           //找到了序号i-1的结点p
   {  s->next=p->next;                    //在p结点后面插入s结点
      p->next=s;
      return true;                        //插入成功返回true
   }
   else                                   //没有找到序号i-1的结点
      return false;                       //参数i错误返回false
}
```

（8）在线性表中删除第i个数据元素Delete(i)



先查找第i-1个结点p：

① 如果存在结点p，若存在后继结点q，删除结点q，返回true；若不存在后继结点q，返回false

② 如果不存在结点p，返回false

删除第i个结点算法



```
bool Delete(int i)                    //在单链表中删除序号i位置的结点
{  if (i<0) return false;             //参数i错误返回false
   LinkNode<T>* p=geti(i-1);          //查找序号i-1的结点
   if (p!=NULL)                       //找到了序号i-1的结点p
   {  LinkNode<T>* q=p->next;         //q指向序号i的结点(被删结点)
      if (q!=NULL)                    //存在序号i的结点时删除它
      {  p->next=q->next;             //删除p结点的后继结点
         delete q;                    //释放空间
         return true;                 //删除成功返回true
      }
      else                            //没有找到序号i的结点
         return false;                //参数i错误返回false
   }
   else                               //没有找到序号i-1的结点
      return false;                   //参数i错误返回false
}
```

（9）输出线性表所有元素DispList()

```
void DispList()                          //输出单链表所有结点值
{  LinkNode<T>* p;
   p=head->next;                         //p指向开始结点
   while (p!=NULL)                       //p不为NULL,输出p结点的data域
   {  cout << p->data << " ";
      p=p->next;                         //p移向下一个结点
   }
   cout << endl;
}
```

2.3.3 单链表的应用算法设计示例

1．基于单链表基本操作的算法设计

【例2.6】有一个长度大于2的整数单链表L，设计一个算法查找L中中间位置的元素。

例如，L=（1，2，3），返回元素为2；L=（1，2，3，4），返回元素为2。

计数法：计算出L的长度*n*，假设首结点的编号为1，则满足题目要求的结点的编号为(*n*-1)/2+1（这里的除法为整除）。置*j*=1，指针*p*指向首结点，让其后移(*n*-1)/2-1个结点即可。

```
template <typename T>
T Middle1(LinkList<T>& L)                    //求解算法1
{  int j=1;
   int n=L.Getlength();
   LinkNode<T>* p=L.head->next;              //p指向首结点
   while (j<=(n-1)/2)                        //找中间位置的p结点
   {  j++;
      p=p->next;
   }
   return p->data;
}
```

快慢指针法：设置快慢指针fast和slow，首先均指向首结点，当fast结点后面至少存在两个结点时，让慢指针slow每次后移动一个结点，让快指针fast每次后移动两个结点。否则slow指向的结点就是满足题目要求的结点。

```
template <typename T>
T Middle2(LinkList<T>& L)                    //求解算法2
{  LinkNode<T>* slow=L.head->next;
   LinkNode<T>* fast=L.head->next;   //均指向首结点
   while (fast!=NULL && slow!=NULL)
   {  if (fast->next==NULL || fast->next->next==NULL)
         return slow->data;                  //满足结束条件时返回
      slow=slow->next;                       //慢指针每次后移1个结点
      fast=fast->next->next;                 //快指针每次后移2个结点
   }
}
```

【例2.7】有一个整数单链表L，其中可能存在多个值相同的结点，设计一个算法查找L中最大值结点的个数。



先让*p*指向首结点，用maxe记录首结点值，将其看成是最大值结点，cnt置为1。按以下方式循环直到*p*指向尾结点为止：

（1）若p->next.data>maxe，将p->next看成新的最大值结点，置maxe=p->next->data，cnt=1。

（2）若p->next->data==maxe，maxe仍为最大结点值，置cnt++。

（3）*p*后移一个结点。

最后返回cnt即为最大值结点个数。

```
template <typename T>
int Maxcount(LinkList<T>& L)                    //求解算法
{  int cnt=1;
   LinkNode<T>* p=L.head->next;                 //p指向首结点
   T maxe=p->data;                              //maxe置为首结点值
   while (p->next!=NULL)                        //循环到p结点为尾结点
   {  if (p->next->data>maxe)                   //找到更大的结点
      {  maxe=p->next->data;                    //第一次找到maxe的结点
         cnt=1;
      }
      else if (p->next->data==maxe)             //p结点为当前最大值结点
         cnt++;                                 //次数增1
      p=p->next;
   }
   return cnt;
}
```

【例2.8】有一个整数单链表L，其中可能存在多个值相同的结点，设计一个算法删除L中所有最大值的结点。



过程如下：

- 先遍历L的所有结点，求出最大结点值maxe。
- 再遍历一次删除所有值为maxe的结点，在删除中，通过（pre，*p*）一对指针指向相邻的两个结点，若p->data==maxe，再通过pre结点删除*p*结点。

```
template <typename T>
void Delmaxnodes(LinkList<T>& L)                    //求解算法
{  LinkNode<T>* p=L.head->next;                     //p指向首结点
   T maxe=p->data;                                  //maxe置为首结点值
   while (p->next!=NULL)                            //查找最大结点值maxe
   {  if (p->next->data>maxe)
         maxe=p->next->data;
      p=p->next;
   }
   LinkNode<T>* pre=L.head;                         //pre指向头结点
   p=pre->next;                                     //p指向pre的后继结点
   while (p!=NULL)                                  //p遍历所有结点
   {  if (p->data==maxe)                            //p结点为最大值结点
      {  pre->next=p->next;                         //删除p结点
         delete p;
         p=pre->next;                               //让p指向pre的后继结点
      }
      else
      {  pre=pre->next;                             //pre后移一个结点
         p=pre->next;                               //让p指向pre的后继结点
      }
   }
}
```

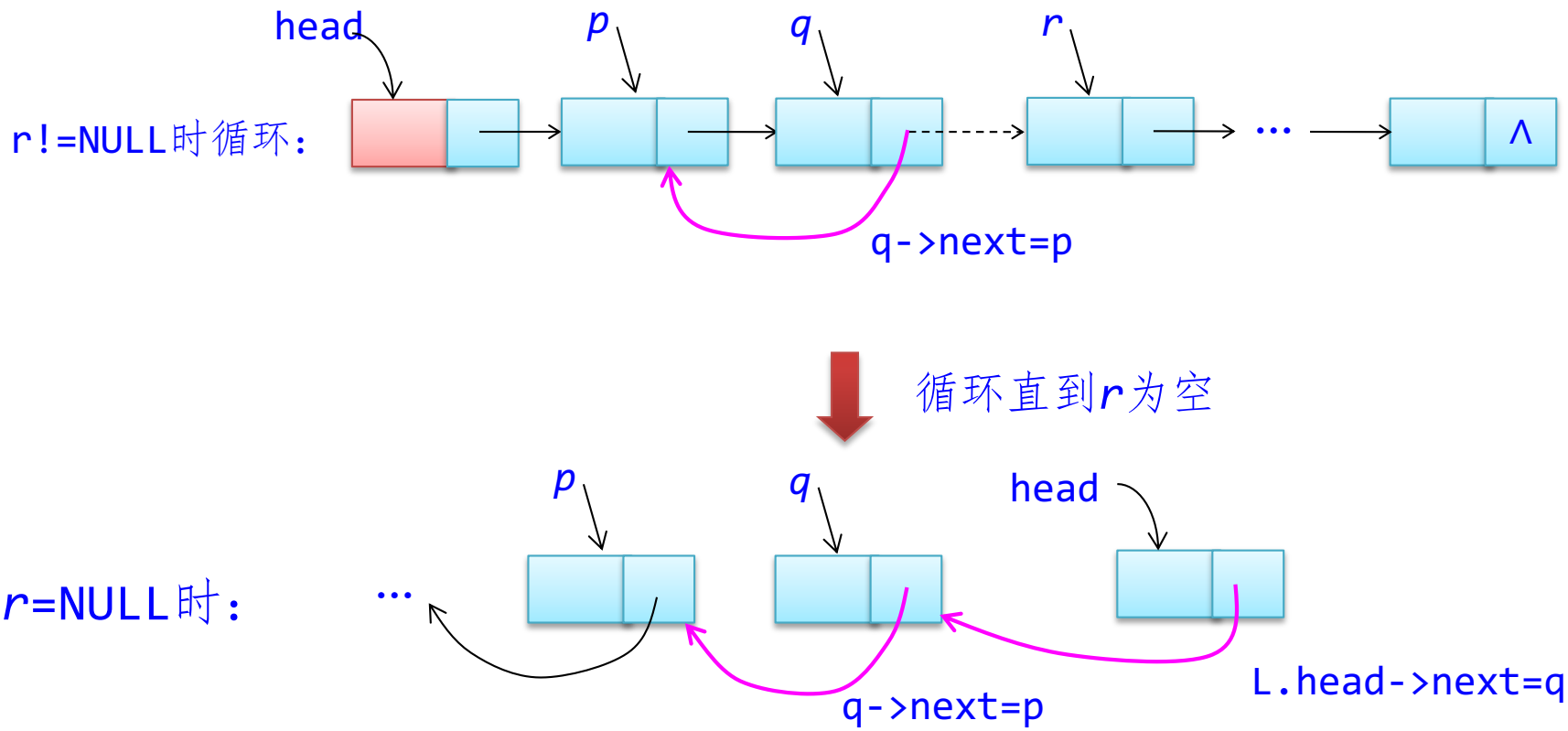
2．基于整体建立单链表的算法设计

【例2.9】有一个整数单链表L，设计一个算法逆置L中的所有结点。例如L=(1，2，3，4，5)，逆置后L=(5，4，3，2，1)。

解法1：采用头插法建表的思路

```
template <typename T>
void Reverse1(LinkList<T>& L)          //求解算法1
{  LinkNode<T>* p=L.head->next,*q;     //p指向首结点
   L.head->next=NULL;                  //将L置为一个空表
   while (p!=NULL)                     //用p遍历所有数据结点
   {  q=p->next;                       //q临时保存p结点的后继结点
      p->next=L.head->next;            //将p结点插入到表头
      L.head->next=p;
      p=q;
   }
}
```

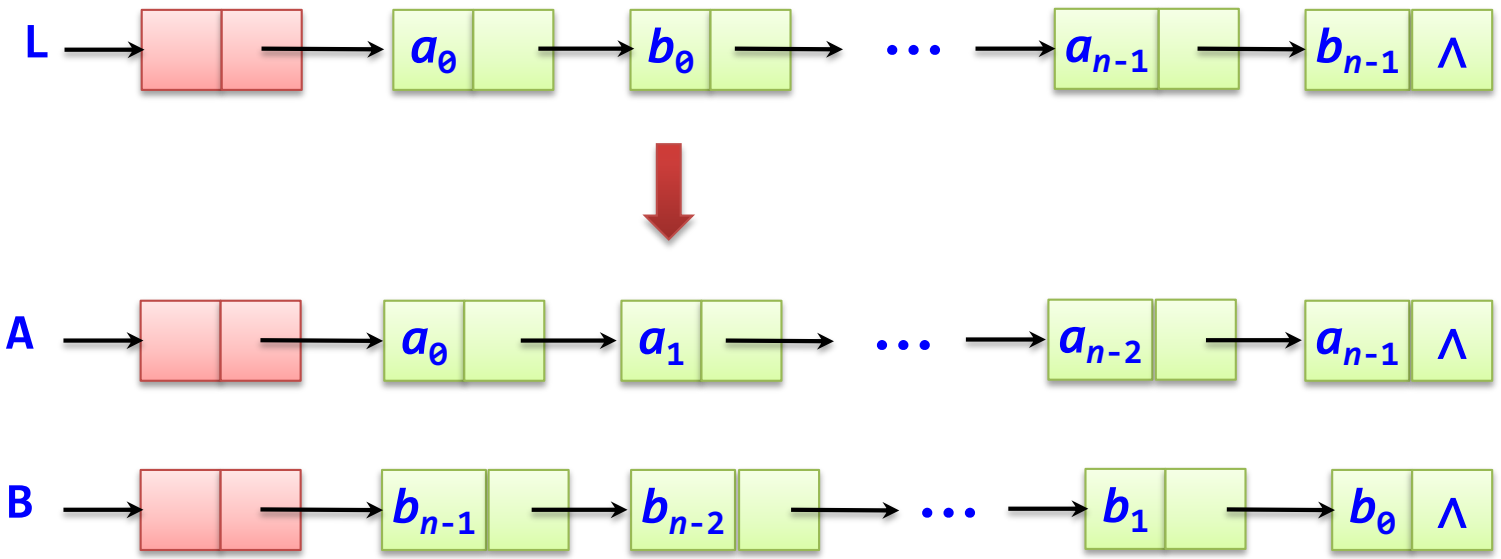
解法2：3指针方法


head
p
q
r
r!=NULL时循环：
…
∧
q->next=p
循环直到r为空
r=NULL时：
p
q
head
…
q->next=p
L.head->next=q

```
template <typename T>
void Reverse2(LinkList<T>& L)          //求解算法2
{  LinkNode<T>* p=L.head->next,*q,*r;//p指向首结点
   if (p==NULL) return;                //L为空单链表时返回
   q=p->next;
   if (q==NULL) return;                //L只有1个结点时返回
   r=q->next;                          //r指向第3个结点
   if (r==NULL)                        //L只有2个结点时
   {  L.head->next=q;
      q->next=p;
      p->next=NULL;
      return;
   }

   p->next=NULL;                       //原首结点p置为尾结点
   while (r!=NULL)                     //(p,q,r)指向的结点都存在时循环
   {  q->next=p;                       //修改结点q的next指针
      p=q;                             //(p,q,r)同步指针后移
      q=r;
      r=r->next;
   }
   q->next=p;                          //修改结点q的next指针
   L.head->next=q;
}
```

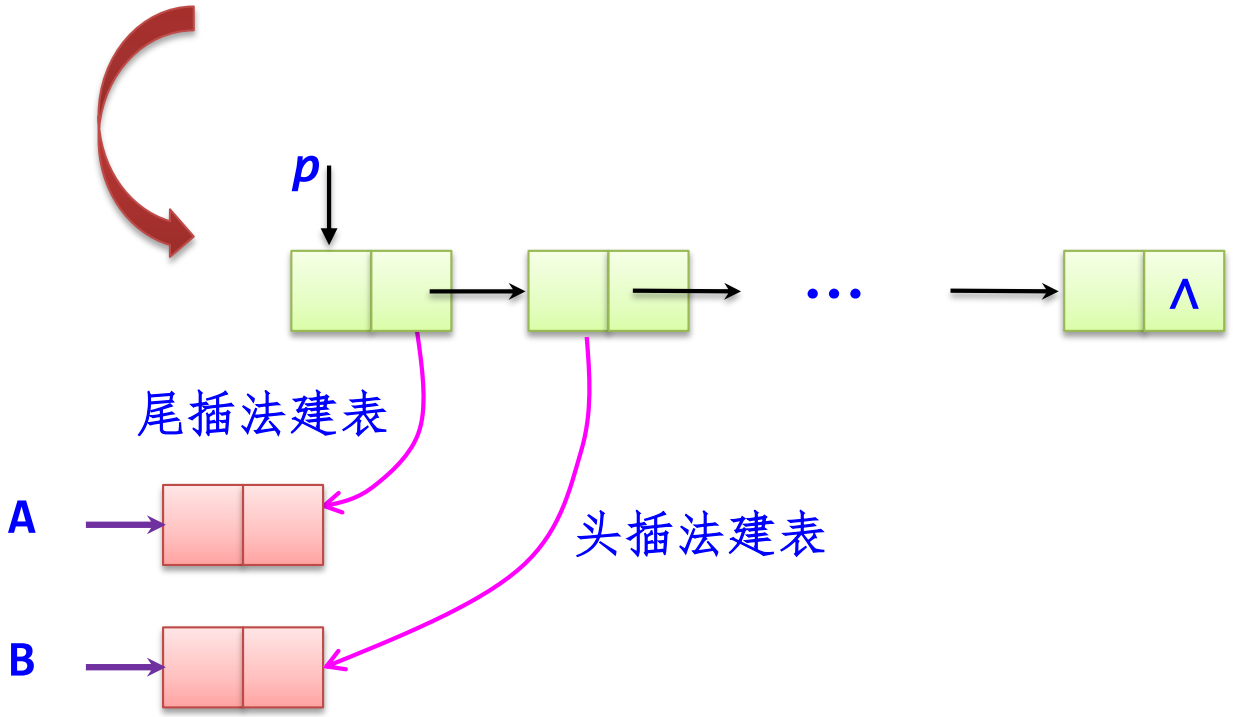
【例2.10】有一个含2n个整数的单链表L=（a_0，b_0，a_1，b_1，…，a_{n-1}，b_{n-1}），设计一个算法将其拆分成两个带头结点的单链表A和B。其中A=（a_0，a_1，…，a_{n-1}），B=（b_{n-1}，b_{n-2}，…，b_0）。



思路

$L=(a_0, b_0, a_1, b_1, \cdots, a_{n-1}, b_{n-1})$

$A=(a_0, a_1, \cdots, a_{n-1}), \quad B=(b_{n-1}, b_{n-2}, \cdots, b_0)$



```
template <typename T>
void Split(LinkList<T>& L,LinkList<T>& A,LinkList<T>& B)
{  LinkNode<T>* p=L.head->next,*q;   //p指向L的首结点
   LinkNode<T>* r=A.head;            //r始终指向A的尾结点

   while (p!=NULL)                   //遍历L的所有数据结点
   {  r->next=p;
      r=p;                           //尾插法建立A
      p=p->next;                     //p后移一个结点
      if (p!=NULL)
      {  q=p->next;                  //临时保存p结点的后继结点
         p->next=B.head->next;       //头插法建立B
         B.head->next=p;
         p=q;                        //p指向q结点
      }
   }
   r->next=NULL;                     //尾结点next置空
}
```

练习

1. 已知一个带有表头结点的单链表，结点定义如下：

```
typedef struct LNode     //定义单链表结点类型
{    ElemType data;
     struct LNode *next;  //指向后继结点
} LinkNode;
```

假设只给出了该链表的头指针 LinkNode *L，请设计一个算法查找链表中**倒数**第 k 个位置的结点($k \geq 0$)，**要求只遍历链表一遍**。如果查找成功，算法返回结果 1 和结点 data 域的值（通过返回型参数传递）；否则，只返回结果 0。

要求：

1） 给出算法的基本设计思想。

2） 根据设计思想，采用 C/C++语言描述算法，对关键的局部变量的使用意义进行注释。

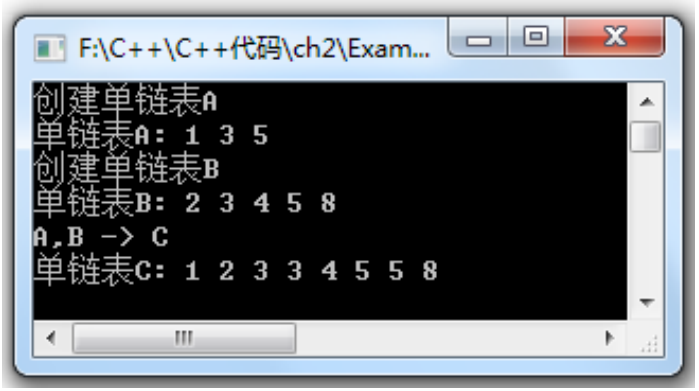
查找结点函数声明如下：

```
int search_k (LinkNode *L, int k, ElemType &e);
```

3．有序单链表的算法设计

【例2.11】有两个递增有序整数单链表A和B，设计一个算法采用二路归并方法将A和B的所有数据结点合并到递增有序单链表C中。

要求算法的空间复杂度为O(1)。



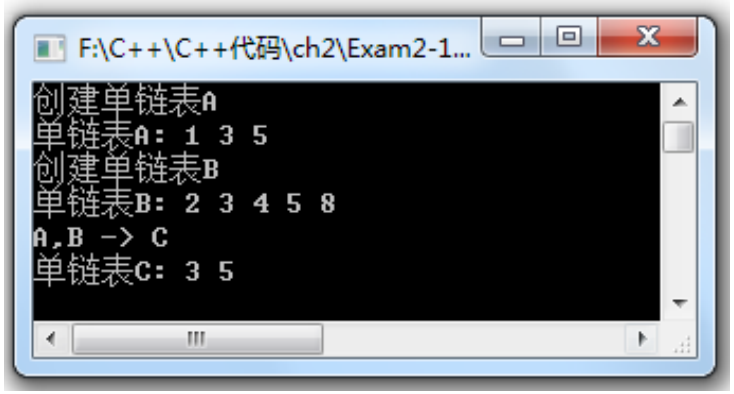
算法的空间复杂度为O(1)?

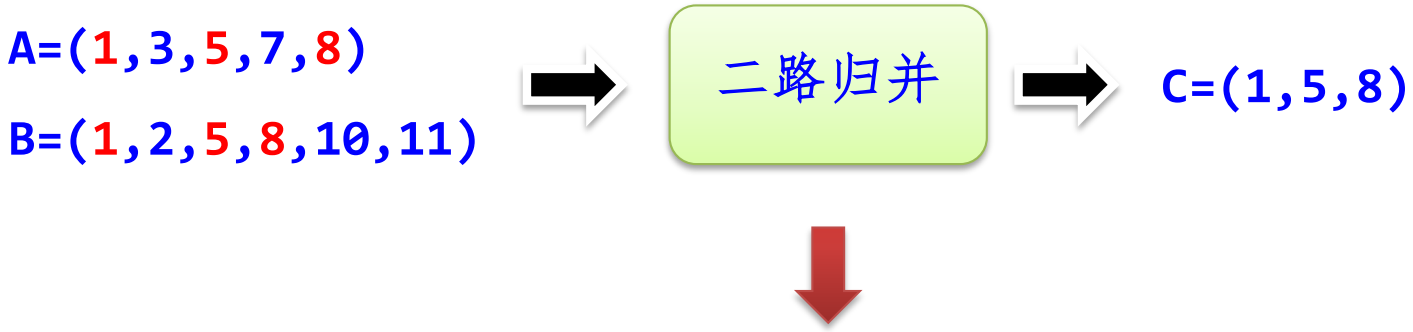
```
template <typename T>
void Merge2(LinkList<T>& A,LinkList<T>& B,LinkList<T>& C)
{  LinkNode<T>* p=A.head->next;        //p指向A的首结点
   LinkNode<T>* q=B.head->next;        //q指向B的首结点
   LinkNode<T>* r=C.head;              //r为C的尾结点

   while (p!=NULL && q!=NULL)          //两个单链表都没有遍历完
   {  if (p->data<q->data)             //将较小结点p链接到C的末尾
      {  r->next=p;
         r=p;
         p=p->next;
      }
      else                             //将较小结点q链接到C的末尾
      {  r->next=q;
         r=q;
         q=q->next;
      }
   }

   r->next=NULL;                       //尾结点next置空
   if (p!=NULL) r->next=p;             //将未归并完的结点链接到C的末尾
   if (q!=NULL) r->next=q;
}
```

【例2.12】有两个递增有序整数单链表A和B，假设每个单链表中没有值相同的结点，但两个单链表中存在相同值的结点，设计一个尽可能高效的算法建立一个新的递增有序整数单链表C，其中包含A和B相同值的结点，要求算法执行后不改变单链表A和B。




A=(1,3,5,7,8)
B=(1,2,5,8,10,11)
二路归并
C=(1,5,8)
二路归并 + 尾插法新建单链表C

```
template <typename T>
void Commnodes(LinkList<T>& A,LinkList<T>& B,LinkList<T>& C)
{  LinkNode<T>* p=A.head->next;                    //p指向A的首结点
   LinkNode<T>* q=B.head->next;                    //q指向B的首结点
   LinkNode<T>* r=C.head;                          //r为C的尾结点

   while (p!=NULL && q!=NULL)                      //两个单链表都没有遍历完
   {  if (p->data<q->data)                         //跳过较小的p结点
         p=p->next;
      else if (q->data<p->data)                    //跳过较小的q结点
         q=q->next;
      else                                         //p结点和q结点值相同
      {  LinkNode<T>* s=new LinkNode<T>(p->data);
         r->next=s;
         r=s;                                      //将s结点链接到C的末尾
         p=p->next;
         q=q->next;
      }
   }
   r->next=NULL;                                   //尾结点next置空
}
```

- 本算法的时间复杂度为O(*n*+*m*)。
- 空间复杂度为O(MIN(*n*，*m*))。
- 其中*m*、*n*分别为A、B单链表中的数据结点个数，MIN为取最小值函数，因为单链表C中最多只有MIN(*n*，*m*)个结点。



逻辑结构特征+基本运算 ⇨ 逻辑特征：元素为一对一关系
基本运算：查找、插入和删除等

存储结构 ⇨ 顺序表，单链表、双链表和循环链表

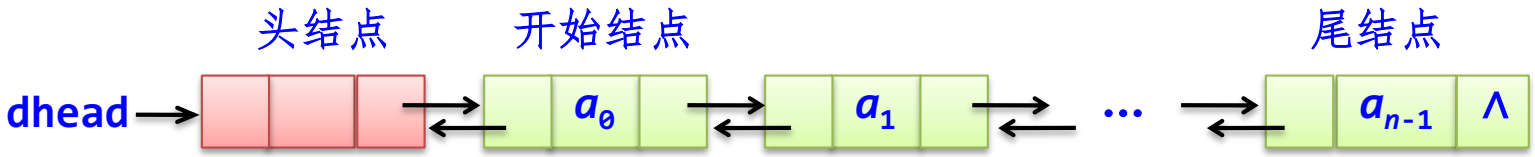
存储结构的基本操作 ⇨ 单链表：指针*p*后移，在*p*结点之后插入一个结点，删除结点*p*的后继结点

基本运算算法设计 ⇨ 头插法、尾插法、查找、插入和删除等算法设计

应用算法设计 ⇨ 单链表逆置，有序单链表二路归并，…

复杂算法设计 ⇨ 两个单链表的公共结点，两个有序单链表的公共结点，3个有序单链表的归并，…

2.3.4 双 链 表



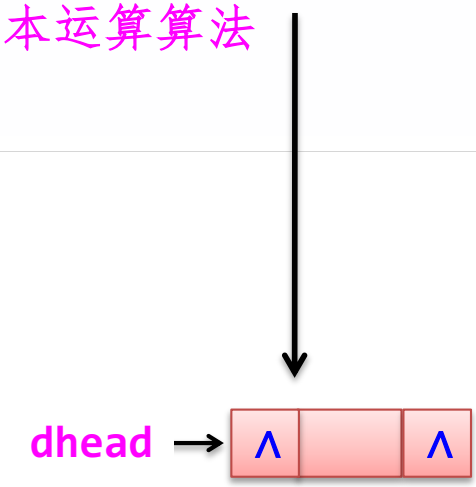
每个结点为DLinkNode类型，包括存储元素的列表data、存储前驱结点指针prior和后继结点指针next。

DLinkNode类型

```
template <typename T>
struct DLinkNode                                    //双链表结点类型
{   T data;                                         //存放数据元素
    DLinkNode<T>* next;                             //指向后继结点的指针
    DLinkNode<T>* prior;                            //指向前驱结点的指针
    DLinkNode():next(NULL),prior(NULL) {}                   //构造函数
    DLinkNode(T d):data(d),next(NULL),prior(NULL) {}  //重载构造函数
};
```

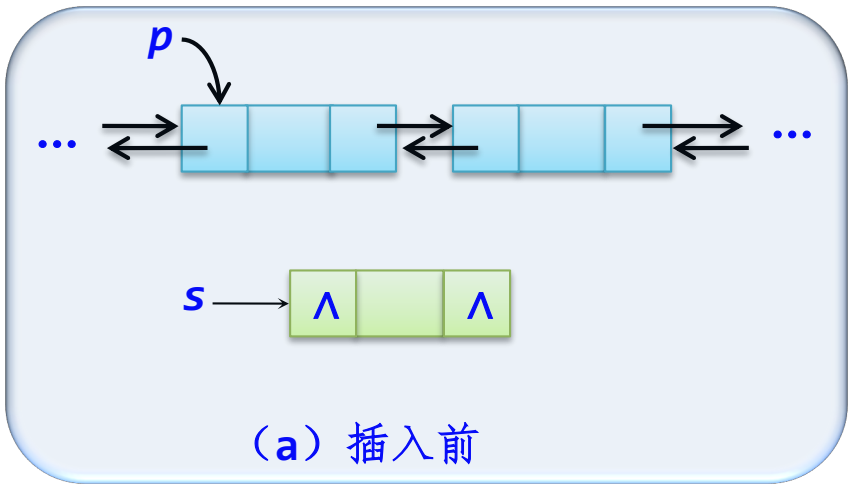
双链表类模板DLinkList

```
template <typename T>
class DLinkList                          //双链表类模板
{
public:
   DLinkNode<T>* dhead;                  //双链表头结点
   //基本运算算法
};
```

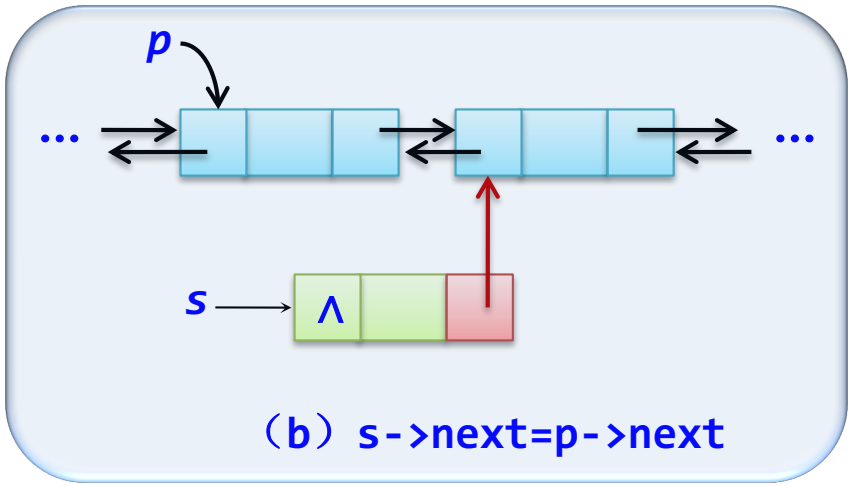


1．插入和删除结点操作

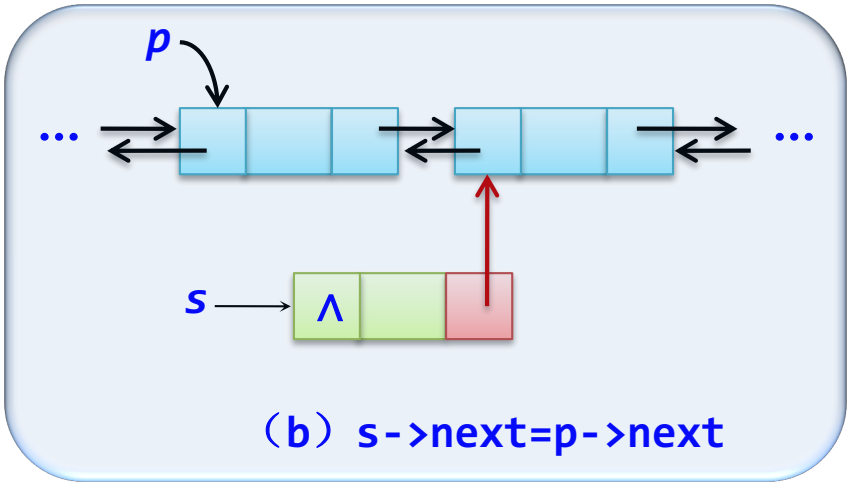
插入结点操作：将结点s插入到双链表中p结点的后面。



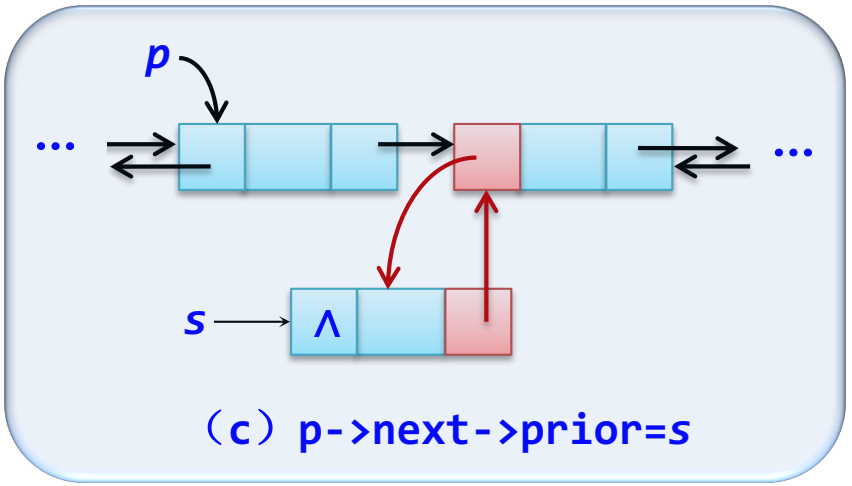
（a）插入前



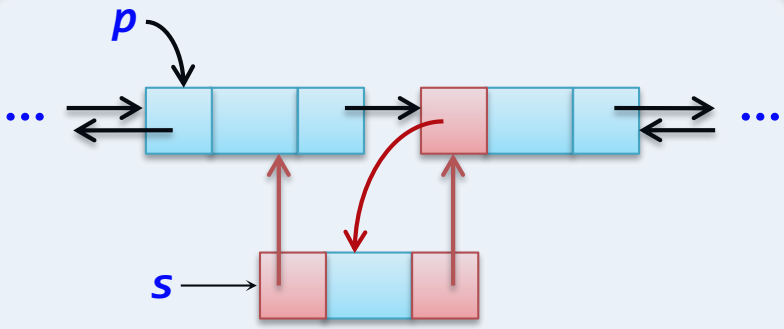
（b）s->next=p->next



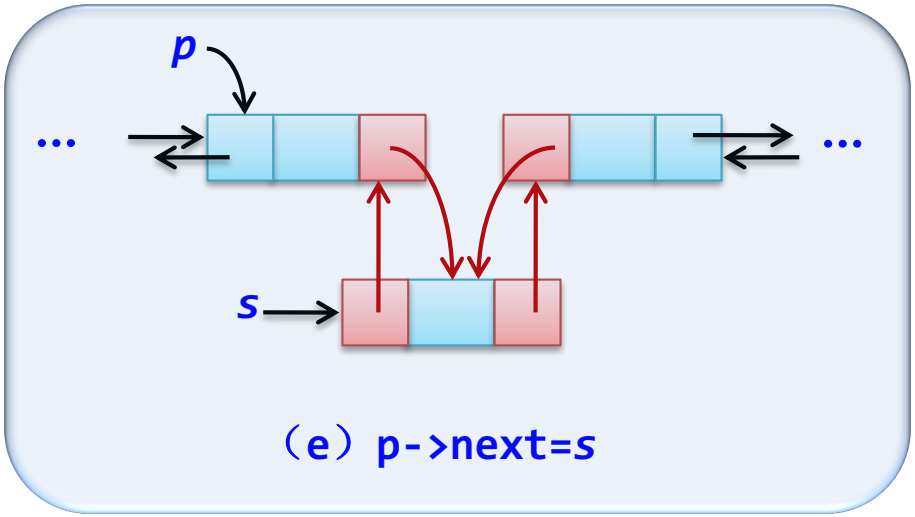
（b）s->next=p->next



（c）p->next->prior=s



（d）s->prior=p

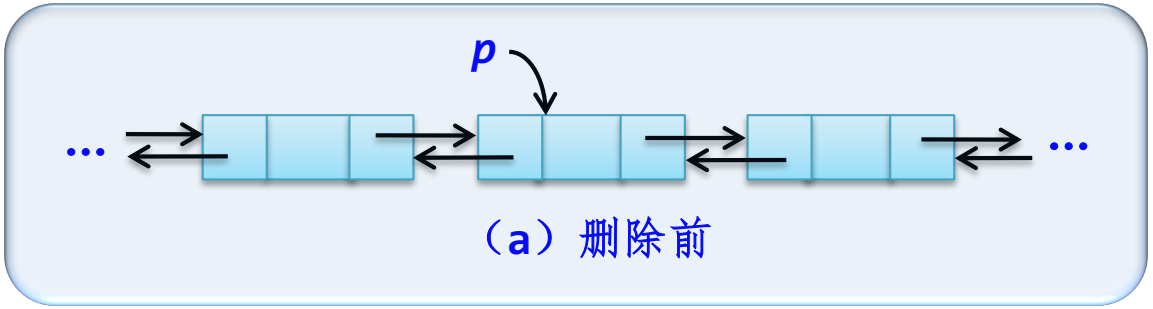


（e）p->next=s

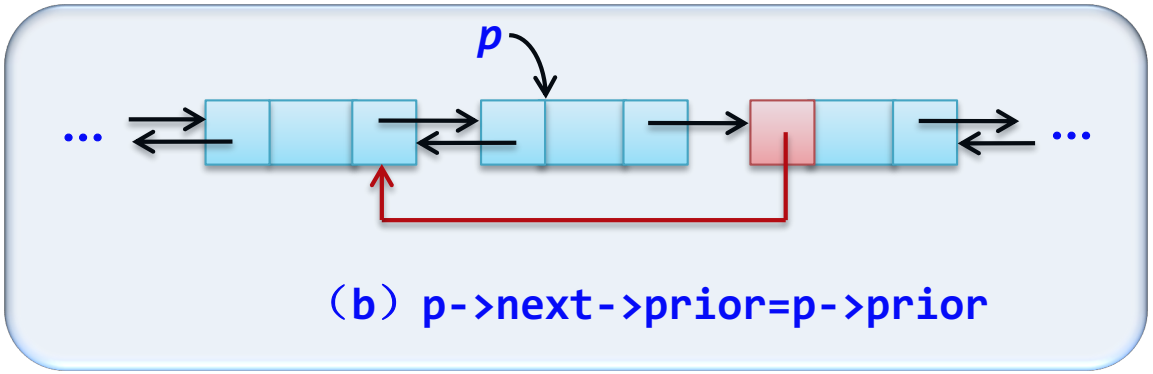
说明

尽可能让间接结点指针修改靠前执行！

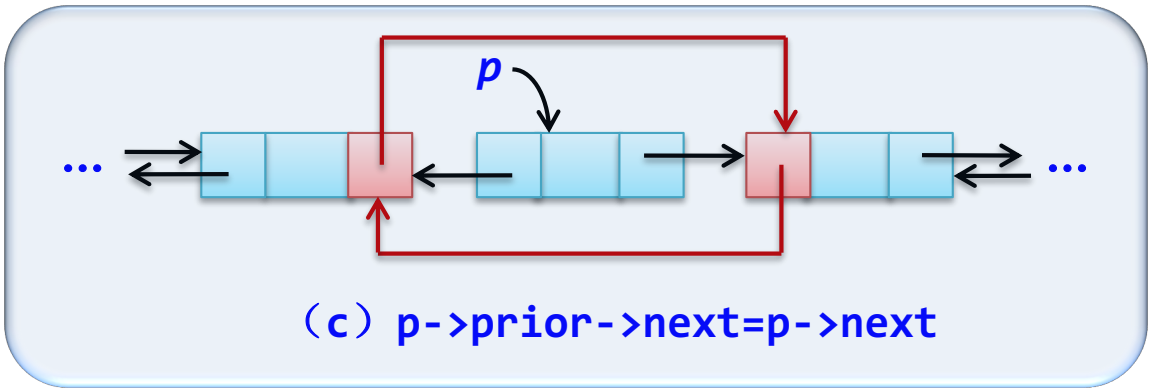
删除结点操作：删除双链表中的*p*结点。



（a）删除前



（b）p->next->prior=p->prior



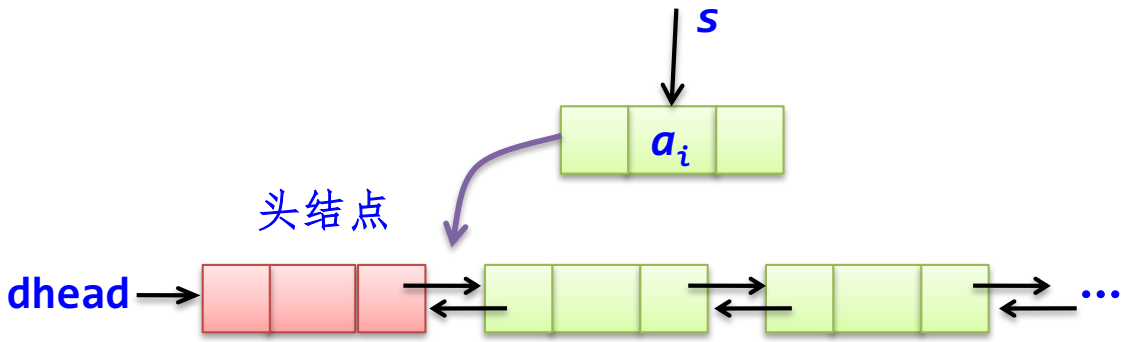
（c）p->prior->next=p->next

2．整体建立双链表

- 通过一个含有*n*个元素的*a*数组来建立双链表。
- 建立双链表的常用方法有两种：头插法和尾插法。

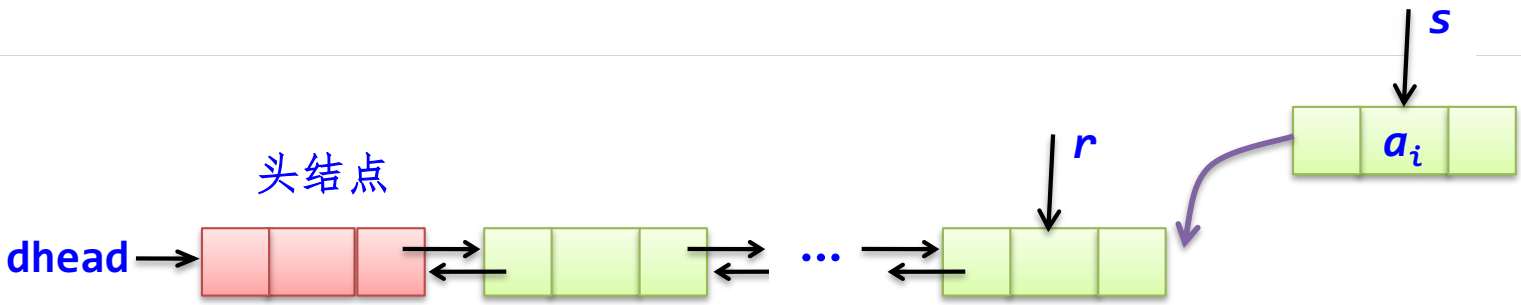
头插法建表

```
void CreateListF(T a[],int n)          //头插法建立双链表
{  DLinkNode<T>* s;
   for (int i=0;i<n;i++)               //循环建立数据结点
   {  s=new DLinkNode<T>(a[i]);        //创建数据结点s
      s->next=dhead->next;             //修改s结点的next成员
      if (dhead->next!=NULL)           //修改头结点的非空后继结点的prior
         dhead->next->prior=s;
      dhead->next=s;                   //修改头结点的next
      s->prior=dhead;                  //修改s结点的prior
   }
}
```



尾插法建表

```
void CreateListR(T a[],int n)          //尾插法建立双链表
{  DLinkNode<T>* s,*r;
   r=dhead;                            //r始终指向尾结点,开始时指向头结点
   for (int i=0;i<n;i++)               //循环建立数据结点
   {  s=new DLinkNode<T>(a[i]);        //创建数据结点s
      r->next=s;                       //将s结点插入r结点之后
      s->prior=r;
      r=s;
   }
   r->next=NULL;                       //将尾结点的next域置为NULL
}
```



3.线性表基本运算在双链表中的实现

- 许多运算算法（如求长度、取元素值和查找元素等）与单链表中相应算法是相同的.
- 涉及结点插入和删除操作的算法需要改为按双链表的方式进行结点插入和删除。

在双链表dhead中序号为i的位置上插入值为e的结点的算法

```
bool Insert(int i,T e)          //在双链表中序号i位置插入值为e的结点
{  if (i<0) return false;                  //参数i错误返回false
   DLinkNode<T>* s=new DLinkNode<T>(e);//建立新结点s
   DLinkNode<T>* p=geti(i-1);              //查找序号i-1的结点p
   if (p!=NULL)                            //找到了序号i-1的结点
   {  s->next=p->next;                     //修改s结点的next域
      if (p->next!=NULL)                   //修改p结点的非空后继结点的prior
         p->next->prior=s;
      p->next=s;                           //修改p结点的next域
      s->prior=p;                          //修改s结点的prior域
      return true;                         //插入成功返回true
   }
   else return false;                      //没有找到序号i-1的结点返回false
}
```



也可以在双链表中找到序号为i的结点p（找后继结点），再在p结点之前插入s结点（后继仅仅之前插入新结点）。

在双链表dhead中删除序号为i的结点的算法

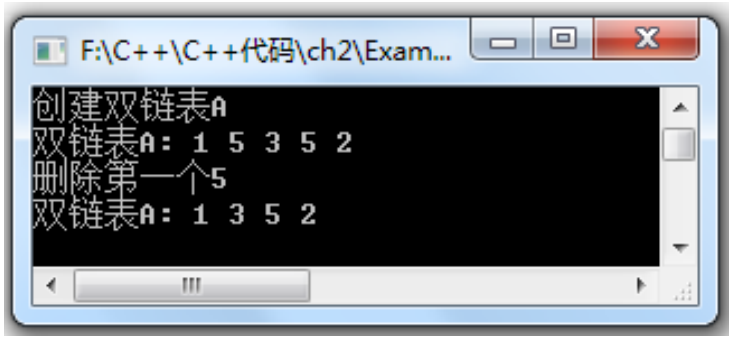
```
bool Delete(int i)                          //在双链表中删除序号i位置的结点
{  if (i<0) return false;                   //参数i错误返回false
   DLinkNode<T>* p=geti(i);                 //查找序号i的结点
   if (p!=NULL)                             //找到了序号i的结点p
   {  p->prior->next=p->next;               //修改p结点的前驱结点的next
      if (p->next!=NULL)                    //修改p结点非空后继结点的prior
         p->next->prior=p->prior;
      delete p;                             //释放空间
      return true;                          //删除成功返回true
   }
   else return false;                       //没有找到序号i-1的结点返回false
}
```



也可以找到序号为i-1的结点p（找前驱结点），再删除其后继结点。

2.3.5 双链表的应用算法设计示例

【例2.13】设计一个算法，删除整数双链表L中第一个值为x的结点，若不存在值为x的结点，则不做任何改变。



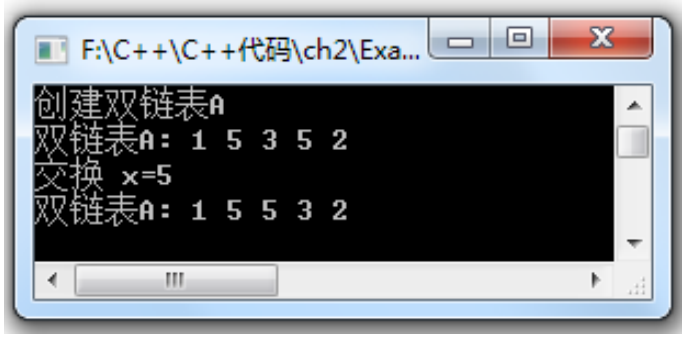


删除整数双链表L中第一个值为x的结点

```
template <typename T>
void Delx(DLinkList<T>& L,T x)                  //求解算法
{  DLinkNode<T>* p=L.dhead->next;               //p指向首结点
   while (p!=NULL && p->data!=x)                //查找第一个值为x的结点
      p=p->next;

   if (p!=NULL)                                 //找到值为x的结点p
   {  p->prior->next=p->next;                   //删除p结点
      if (p->next!=NULL)
         p->next->prior=p->prior;
      delete p;                                 //释放空间
   }
}
```

【例2.14】设计一个算法，将整数双链表L中最后一个值为x的结点与其前驱结点交换。若不存在值为x的结点或者该结点是首结点，则不做任何改变。





将整数双链表L中最后一个值为x的结点与其前驱结点交换

```
template <typename T>
void Swap(DLinkList<T>& L,T x)          //求解算法
{  DLinkNode<T>* p=L.dhead->next,*q; //p指向首结点
   q=NULL;
   while (p!=NULL)                      //查找最后一个值为x的结点q
   {  if (p->data==x) q=p;
      p=p->next;
   }
```

```
if (q==NULL || L.dhead->next==q)      //不存在x结点或者该结点是首结点
      return;                         //直接返回
   else                               //找到了这样的结点q
   {  DLinkNode<T>* pre=q->prior;     //pre指向结点q的前驱结点
      pre->next=q->next;              //删除q结点
      if (q->next!=NULL)
         q->next->prior=pre;
      pre->prior->next=q;             //将q结点插入到pre结点之前
      q->prior=pre->prior;
      pre->prior=q;
      q->next=pre;
   }
}
```

2.3.6 循环链表

循环链表是另一种形式的链式存储结构形式。

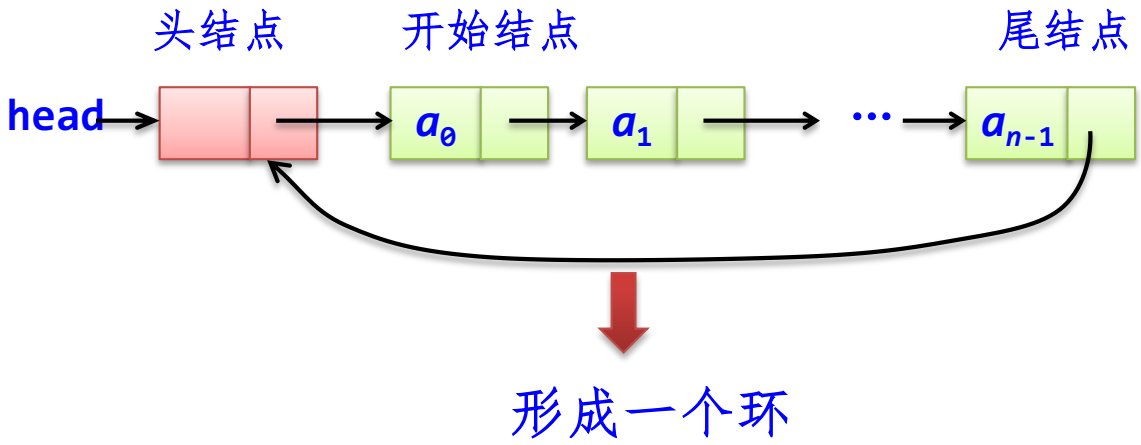
- **循环单链表**：将表中尾结点的指针域改为指向表头结点，整个链表形成一个环。由此从表中任一结点出发均可找到链表中其他结点。
- **循环双链表**：形成两个环。

→ 结点类型与非循环单链表的相同

→ 结点类型与非循环双链表的相同

2.3.6 循环链表

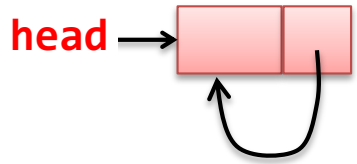
1. 循环单链表



循环单链表类模板CLinkList

```
template <typename T>
class CLinkList                    //循环单链表类
{
public:
  LinkNode<T>* head;               //循环单链表头结点
  CLinkList()                      //构造函数,创建空循环单链表
  {  head=new LinkNode<T>();
     head->next=head;              //构成循环的空链表
  }
  ~CLinkList()                     //析构函数,销毁循环单链表
  {  LinkNode<T>* pre,*p;
     pre=head;p=pre->next;
     while (p!=head)               //用p遍历并释放其前驱结点
     {  delete pre;                //释放pre结点
        pre=p;p=p->next;           //pre,p同步后移一个结点
     }
     delete pre;                   //释放尾结点
   }
   //线性表的基本运算算法
};
```

结点类型与非循环单链表中的结点类型相同



循环单链表的插入和删除结点操作与非循环单链表的相同，所以两者的许多基本运算算法是相似的，主要区别如下：

- 初始只有头结点head，在循环单链表的构造方法中需要通过head->next=head语句置为空表。
- 循环单链表中涉及查找操作时需要修改表尾判断的条件，例如，用*p*遍历时，尾结点满足的条件是p->next==head而不是p->next==NULL。

【例2.15】有一个整数循环单链表L，设计一个算法求值为x的结点个数。




```
template <typename T>
int Count(CLinkList<T>& L,T x)          //求解算法
{  int cnt=0;                           //cnt置为0
   LinkNode<T>* p=L.head->next;         //首先让p指向首结点
   while (p!=L.head)                    //遍历循环单链表
   {  if (p->data==x)
         cnt++;                         //找到一个值为x的结点cnt增1
      p=p->next;                        //p后移一个结点
   }
   return cnt;
}
```

【例2.16】编写一个程序求解约瑟夫（Joseph）问题。有n个小孩围成一圈，给他们从1开始依次编号，从编号为1的小孩开始报数，数到第m（$0<m<n$）个小孩出列，然后从出列的下一个小孩重新开始报数，数到第m个小孩又出列，…，如此反复直到所有的小孩全部出列为止，求整个出列序列。

如当$n=6$，$m=5$时的出列序列是5，4，6，2，3，1。

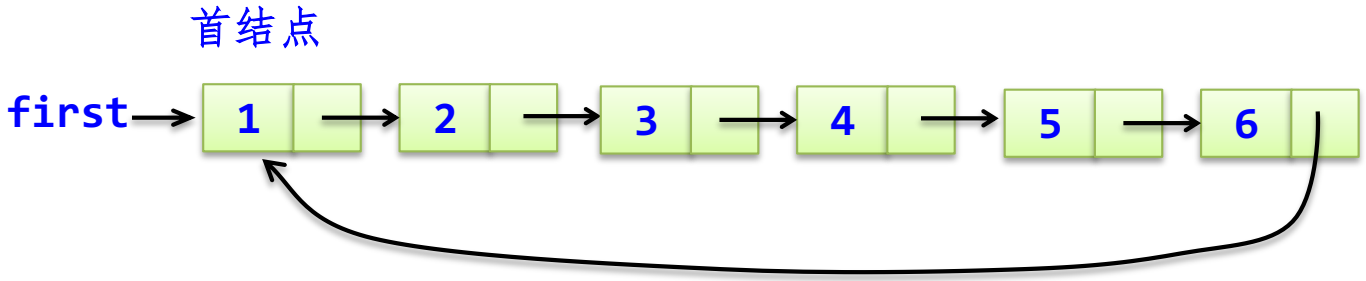


（1）设计存储结构

本题采用不带头结点的循环单链表存放小孩圈，其结点类如下：

```
struct Child                                    //小孩结点类型
{   int no;                                     //小孩编号
    Child* next;                                //下一个结点指针
    Child(int d):no(d),next(NULL) {}            //构造函数
};
```

例如，*n*=6时的初始循环单链表如下图所示，first指向开始报数的小孩结点，初始时指向首结点。



（2）设计基本运算算法

设计一个求解约瑟夫问题的Joseph类，其中包含：

- *n*、*m*整型成员和首结点指针first成员。
- CreateList函数用于建立有*n*个结点的不带头结点的循环单链表first。
- Jsequence函数用于产生约瑟夫序列。

```
class Joseph                                        //求解约瑟夫问题类
{  int n,m;
   Child* first;                                    //小孩循环单链表的首结点
public:
   Joseph(int n1,int m1):n(n1),m(m1) {}     //构造函数
   void CreateList()                                //创建小孩循环单链表
   {  first=new Child(1);                           //循环单链表首结点
      Child* r=first,*p;                            //r为尾结点指针
      for (int i=2;i<=n;i++)
      {  p=new Child(i);                            //建立一个编号为i的新结点p
         r->next=p;                                 //将p结点链到末尾
         r=p;
      }
      r->next=first;                                //构成首结点为first的循环单链表
   }
```

```
    void Jsequence()                    //输出约瑟夫序列
    {  Child* p,*q;
       for (int i=1;i<=n;i++) //共出列n个小孩
       {  p=first;
          int j=1;
          while (j!=m)                  //从first结点开始报数，报到第m个结点
          {  j++;                                   //报数递增
             p=p->next;                             //移到下一个结点
          }
          cout << p->no << " ";                     //该结点的小孩出列
          q=p->next;                                //q指向结点p的后继结点
          p->no=q->no;                              //将结点q的值复制到结点p
          p->next=q->next;                          //删除q结点
          delete q;                                 //释放结点空间
          first=p;                                  //从结点p重新开始
       }
       cout << endl;
    }
};
```

（3）设计主程序

设计如下主程序求解一个约瑟夫序列。

```
int main()
{  int n=6,m=3;
   Joseph L(n,m);
   cout << "n=" << n << ", m=" << m << "的约瑟夫序列:" << endl;
   L.CreateList();
   L.Jsequence();
   return 0;
}
```

（4）执行结果

本程序的执行结果如下：

```
n=6，m=3的约瑟夫序列：
3 6 4 2 5 1
```

【例（补充）】某线性表最常用的操作是在尾元素之后插入一个元素和删除第一个元素，故采用（　）存储方式最节省运算时间。

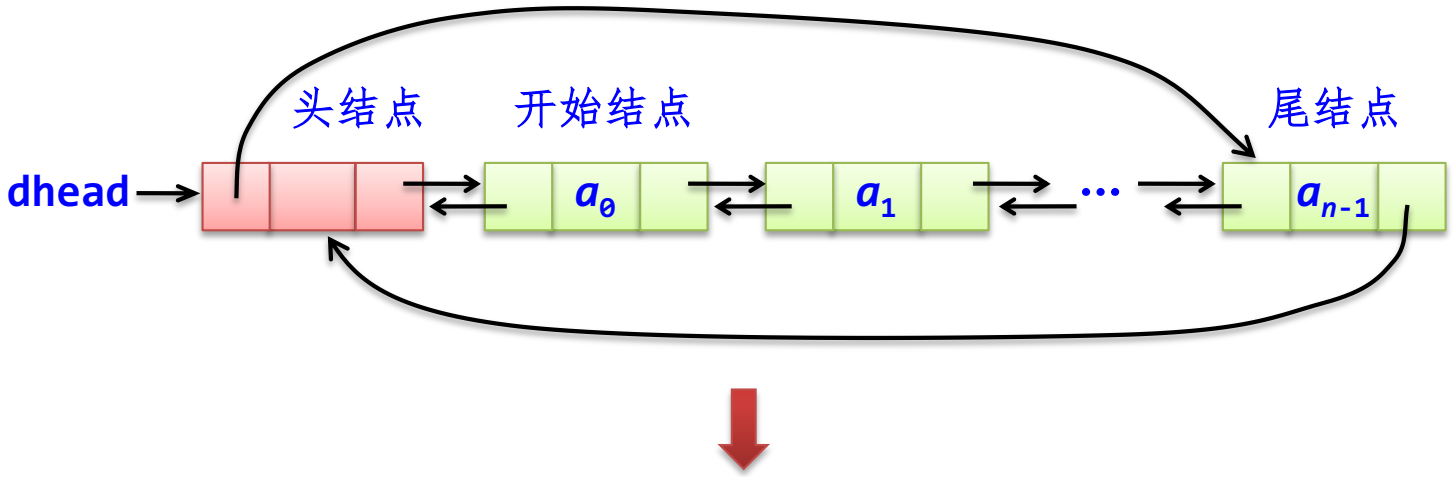
A.单链表

B.仅有头结点指针的循环单链表

C.双链表

D.仅有尾结点指针的循环单链表

2．循环双链表

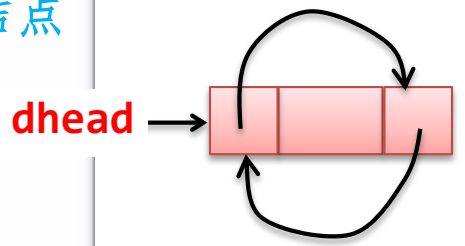


- 形成两个环
- 可以快速找到尾结点

循环双链表类模板DCLinkList

```
template <typename T>
class CDLinkList                        //循环双链表类模板
{
public:
   DLinkNode<T>* dhead;                 //循环双链表头结点
   CDLinkList()                         //构造函数,创建空表
   {  dhead=new DLinkNode<T>();
      dhead->next=dhead;                //构成循环的空链表
      dhead->prior=dhead;
   }
   ~CDLinkList()                        //析构函数,销毁
   {  DLinkNode<T>* pre,*p;
      pre=dhead;p=pre->next;
     while (p!=dhead)                   //p遍历并释放前驱结点
     {  delete pre;                     /、释放pre结点
        pre=p;p=p->next;                //pre,p同步后移一个结点
     }
     delete pre;                        //此时释放尾结点
   }
   //基本运算算法
};
```

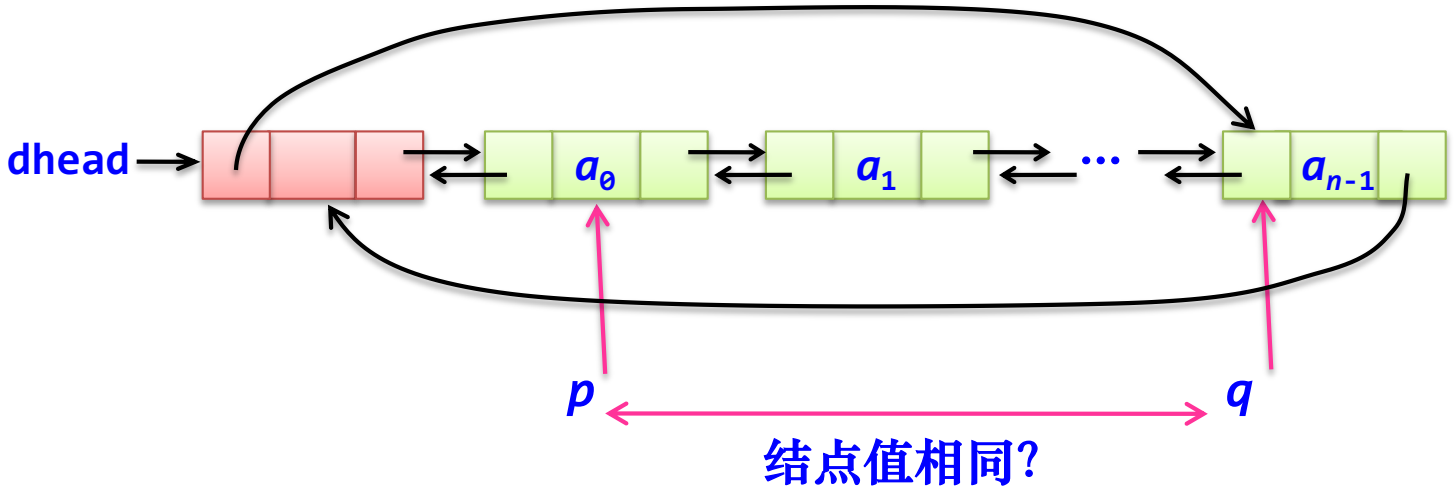
结点类型与非循环双链表中的结点类型相同



循环双链表的插入和删除结点操作与非循环双链表的相同，所以两者的许多基本运算算法是相似的，主要区别如下：

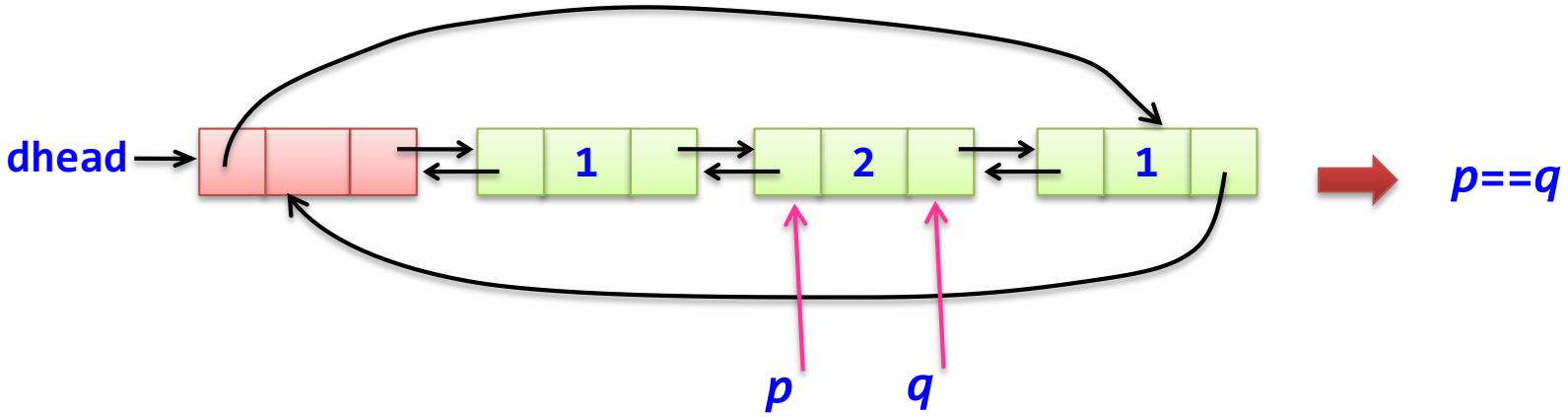
- 初始只有头结点dhead，在循环双链表的构造方法中需要通过dhead->prior=dhead和dhead->next=dhead两个语句置为空表。
- 循环双链表中涉及查找操作时需要修改表尾判断的条件，例如，用*p*遍历时，尾结点满足的条件是p->next==dhead而不是p->next==NULL。

【例2.18】有一个带头结点的循环双链表L，其结点data成员值为整数，设计一个算法，判断其所有元素是否对称。如果从前向后读和从后向前读得到的数据序列相同，表示是对称的；否则不是对称的。

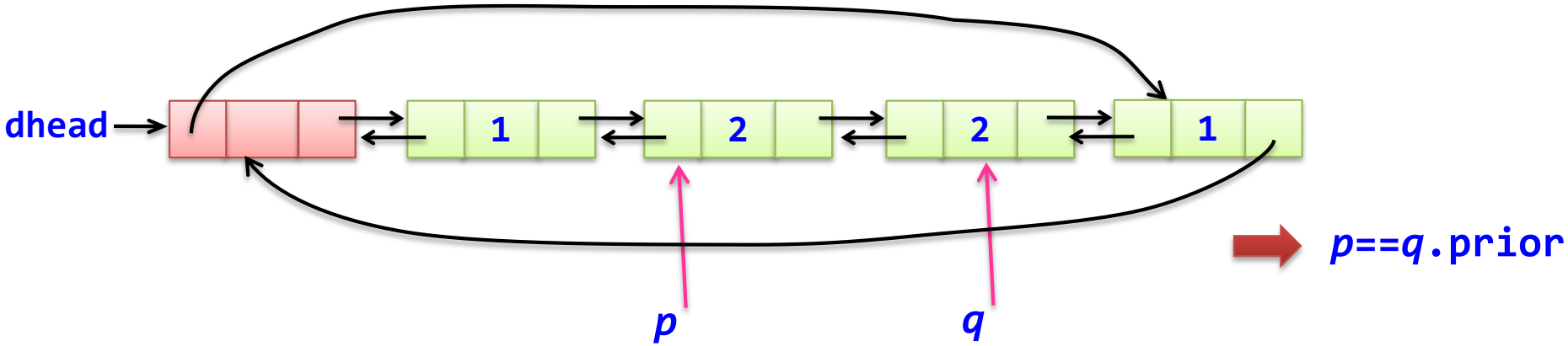


循环结束条件?

（1）结点个数为奇数



（2）结点个数为偶数



```
template <typename T>
bool Symm(CDLinkList<T>& L)                  //求解算法
{  bool flag=true;                           //flag表示L是否对称，初始时为真
   DLinkNode<T>* p=L.dhead->next;            //p指向首结点
   DLinkNode<T>* q=L.dhead->prior;           //q指向尾结点

   while (flag)
   {  if (p->data!=q->data)                  //对应结点值不相同，置flag为假
         flag=false;
      else
      {  if (p==q || p==q->prior)            //满足结束条件退出循环
            break;
         q=q->prior;                         //q前移一个结点
         p=p->next;                          //p后移一个结点
      }
   }
   return flag;
}
```

【例（补充）】如果对含有n（$n>1$）个元素的线性表的运算只有4种，即删除第一个元素、删除尾元素、在第一个元素前面插入新元素、在尾元素的后面插入新元素，则最好使用（　　）。

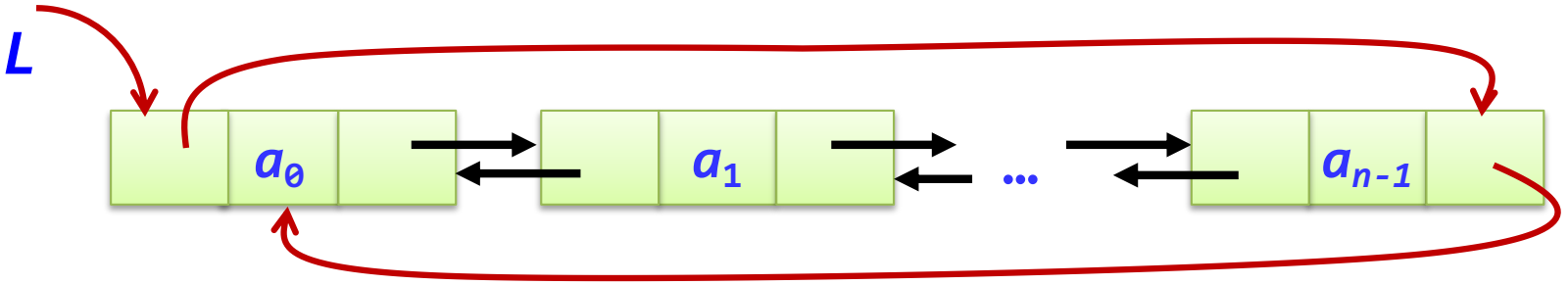
A.只有尾结点指针没有头结点的循环单链表

B.只有尾结点指针没有头结点的非循环双链表

C.只有首结点指针没有尾结点指针的循环双链表

D.既有头指针也有尾指针的循环单链表

只有首结点指针没有尾结点指针的循环双链表



- 删除第一个元素
- 删除尾元素
- 在第一个元素前面插入新元素
- 在尾元素的后面插入新元素

时间复杂度均为O(1)

选择C

2.4 顺序表和链表的比较

1．基于空间的考虑

$$\text{存储密度} = \frac{\text{结点中数据本身占用的存储量}}{\text{整个结点占用的存储量}}$$

一般地，存储密度越大，存储空间的利用率就越高。

- 顺序表的存储密度为1，而链表的存储密度小于1。仅从存储密度看，顺序表的存储空间利用率高。
- 顺序表需要预先分配初始空间，所有数据占用一整片地址连续的内存空间，如果分配的空间过小，易出现上溢出，需要扩展空间导致大量元素移动而降低效率；如果分配的空间过大，会导致空间空闲而浪费。而链表的存储空间是动态分配的，只要内存有空闲，就不会出现上溢出。
- **结论：**当线性表的长度变化不大，易于事先确定的情况下，为了节省存储空间，宜采用顺序表作为存储结构。当线性表的长度变化较大，难以估计其存储大小时，为了节省存储空间，宜采用链表作为存储结构。

2．基于时间的考虑

- 顺序表具有随机存取特性，给定序号查找对应的元素值的时间为O(1)，而链表不具有随机存取特性，只能顺序访问，给定序号查找对应的元素值的时间为O(*n*)。
- 在顺序表中插入和删除操作时，通常需要平均移动半个表的元素。而在链表中插入和删除操作仅仅需要修改相关结点的指针成员，不必移动结点。
- **结论**：若线性表的运算主要是查找，很少做插入和删除操作，宜采用顺序表作为存储结构。若频繁地做插入和删除操作，宜采用链表作为存储结构。

2.5 线性表的应用–两个多项式相加

2.5.1 问题描述

多项式 $p(x) = c_1 x^{e_1} + c_2 x^{e_2} + \cdots + c_m x^{e_m}$



求两个多项式相加的程序

例如，$p(x)=2x^3+3.2x^5-6x+10$，$q(x)=6x+1.8x^5-2x^3+x^2-2.5x^4-5$



$r(x)=p(x)+q(x)$

$r(x)=5x^5-2.5x^4+x^2+5$

两个多项式的数据分别存放在abc1.in和abc2.in文本文件中，要求相加的结果多项式的数据存放在abc.out文本文件中。

abc1.in文件

```
4
2 3
3.2 5
-6 1
10 0
```

$p(x)=2x^3+3.2x^5-6x+10$

abc2.in文件

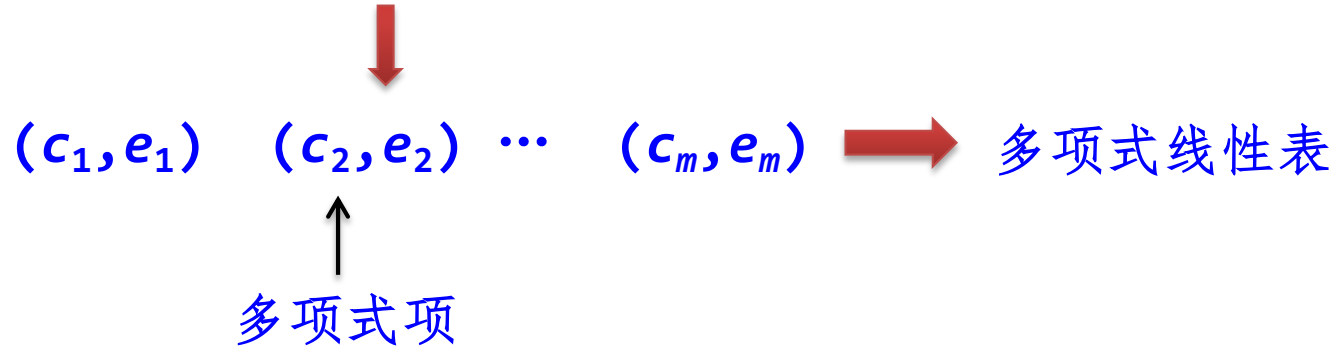
```
6
6 1
1.8 5
-2 3
1 2
-2.5 4
```

$q(x)=6x+1.8x^5-2x^3+x^2-2.5x^4-5$

abc.out文件

```
第1个多项式:    [[2.0, 3],[3.2, 5],[-6.0, 1],[10.0, 0]]
排序后结果:     [[3.2, 5],[2.0, 3],[-6.0, 1],[10.0, 0]]
第2个多项式:    [[6.0, 1],[1.8, 5],[-2.0, 3],[1.0, 2],[-2.5, 4],[-5.0, 0]]
排序后结果:     [[1.8, 5],[-2.5, 4],[-2.0, 3],[1.0,2],[6.0, 1], [-5.0, 0]]
相加多项式:     [[5.0, 5],[-2.5, 4],[1.0, 2],[5.0, 0]]
```

$$p(x) = c_1 x^{e_1} + c_2 x^{e_2} + \cdots + c_m x^{e_m}$$



ADT PolyClass　　//多项式抽象数据类型

{　数据对象：

　　PolyElem={(c_i, e_i) | $1 \leqslant i \leqslant n$, $c_i \in$float, $e_i \in$int};

　数据关系：

　　r={<x_i, y_i> | x_i, $y_i \in$PolyElem, i=1, …, n-1}

　基本运算：

　　初始化和销毁：分别用于建立空存储结构和释放其空间。

　　CreateList(fname)：从fname文件中读取数据建立多项式。

　　Sort()：对多项式按指数递减排序。

　　DispPoly()：输出多项式存储结构。

} //ADT PolyClass

2.5.2 问题求解

1．设计链式存储结构

- 一个多项式用一个带头结点的单链表存储，每个结点存储一个多项式项$[c_i, e_i]$（其中c_i为系数，e_i为指数）

多项式结点类型PolyNode

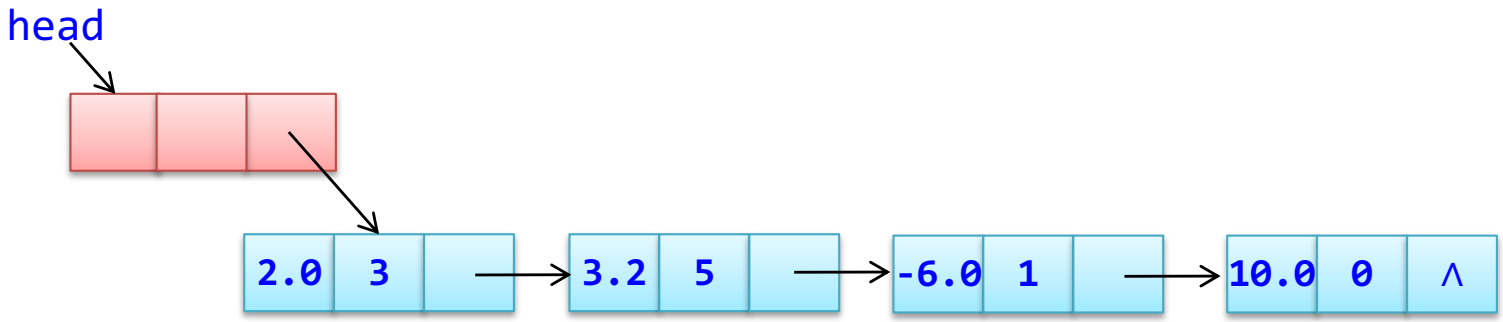
```
struct PolyNode                          //多项式单链表结点类型
{  double coef;                          //系数
   int exp;                              //指数
   PolyNode* next;                       //指向下一个结点的指针

   PolyNode():next(NULL) {}              //构造函数
   PolyNode(double c,int e)              //重载构造函数
   {  coef=c;
      exp=e;
      next=NULL;
   }
};
```

- 设计多项式单链表类为PolyList，其中构造函数和析构函数的设计思路与2.3.2节单链表的完全相同。PolyList类的定义如下：

```
class PolyList                          //多项式单链表类
{
public:
   PolyNode* head;                      //多项式单链表的头结点指针
   PolyList()                           //构造函数
   {
      head=new PolyNode();              //建立头结点
   }
   ~PolyList()                          //析构函数
   {  PolyNode* pre=head,*p=pre->next;
      while (p!=NULL)
      {  delete pre;
         pre=p; p=p->next;              //pre、p指针同步后移
      }
      delete pre;
   }
   void CreateList(char* fname); //读文件采用尾插法建立多项式单链表
   void Sort();                         //对多项式单链表按exp域递减排序
   void DispPoly();                     //输出多项式单链表
};
```

多项式$p(x)=2x^3+3.2x^5-6x+10$对应的单链表：



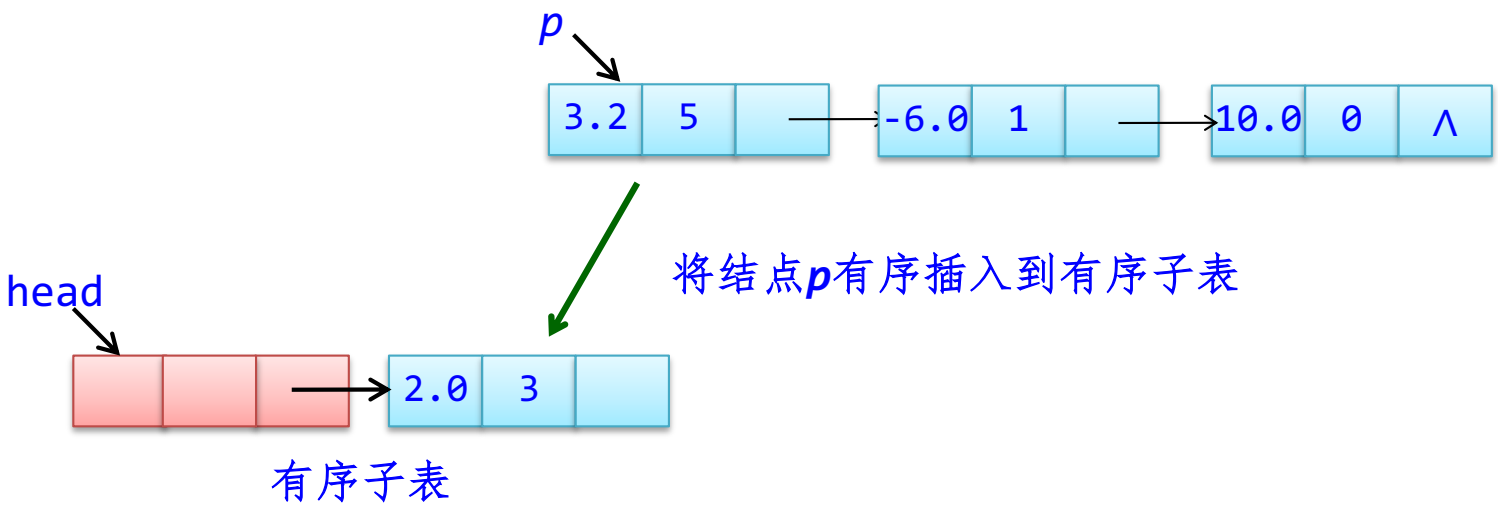
2．设计PolyList的基本运算算法

（1）创建多项式单链表CreateList(fname)

```
void CreateList(char* fname) //读文件采用尾插法建立多项式单链表
{  freopen(fname,"r",stdin); //输入重定向到fname文件
   PolyNode* s,*r;
   double c;
   int n,e;
   scanf("%d",&n);
   r=head;                   //r始终指向尾结点,开始时指向头结点
   for (int i=0;i<n;i++)
   {  scanf("%lf%d",&c,&e);
      s=new PolyNode(c,e);   //创建新结点s
      r->next=s;             //将结点s插入结点r之后
      r=s;
   }
   r->next=NULL;             //尾结点next域置为NULL
}
```

读文件+尾插法建表

（2）多项式单链表排序Sort()



```
void Sort()                          //对多项式单链表按exp域递减排序
{  PolyNode* p,*pre,*q;
   q=head->next;                     //q指向开始结点
   if (q==NULL) return;              //原单链表空时返回
   p=head->next->next;               //p指向结点q的后继结点
   if (p==NULL) return;              //原单链表只有一个数据结点时返回

   q->next=NULL;                     //构造只含一个数据结点的有序单链表

   while (p!=NULL)
   {  q=p->next;                     //q用于临时保存结点p后继结点

      pre=head;                      //从有序表开头比较
      while (pre->next!=NULL && pre->next->exp > p->exp)
         pre=pre->next;              //在有序表中查找插入结点p的前驱结点pre
      p->next=pre->next;             //在结点pre之后插入结点p
      pre->next=p;
      p=q;                           //继续处理原单链表余下的结点
   }
}
```

查找到第一个≤p->exp的位置的前驱结点pre

(3) 输出多项式单链表DispPoly()

```
void DispPoly()                                  //输出多项式单链表
{  bool first=true;                              //first为true表示是第一项
   PolyNode* p=head->next;                       //p指向开始结点
   while (p!=NULL)
   {  if (first)
      {  printf("[%.1lf,%d]",p->coef,p->exp);
         first=false;
      }
      else
         printf(",[%.1lf,%d]",p->coef,p->exp);
      p=p->next;
   }
   printf("\n");
}
```

3.设计两个多项式单链表相加运算算法PolyAdd()

两个按指数递减排序的多项式单链表



二路归并 + 尾插法建表

相加的结果多项式单链表

用pa、pb分别遍历A和B的结点，先建立一个空多项式单链表C，在pa、pb都没有遍历完时循环：

① 若pa结点的指数较大，复制pa结点并添加到C的末尾，同时pa后移一个结点。

② 若pb结点的指数较大，复制pb结点并添加到C的末尾，同时pb后移一个结点。

③ 若pa和pb结点的指数相同，求出它们的系数和c（c=pa->coef+pb->coef），如果c≠0，由c和pa->exp新建一个结点并添加到C的末尾，否则不新建结点，pa、pb均后移一个结点。

上述循环过程结束后，若有一个多项式单链表没有遍历完，说明余下的多项式项都是指数较小的多项式项，将它们均复制并添加到C末尾。

```
void PolyAdd(PolyList& A,PolyList& B,PolyList& C)  //A+B->C
{  PolyNode* pa=A.head->next;                      //pa指向A的开始结点
   PolyNode* pb=B.head->next;                      //pb指向B的开始结点
   PolyNode* s,*r;
   double c;
   r=C.head;                                       //r指向尾结点

   while (pa!=NULL && pb!=NULL)
   {  if (pa->exp>pb->exp)                         //归并指数较大的结点pa
      {  s=new PolyNode(pa->coef,pa->exp);         //复制产生结点s
         r->next=s; r=s;                           //将结点s链到C末尾
         pa=pa->next;
      }
```

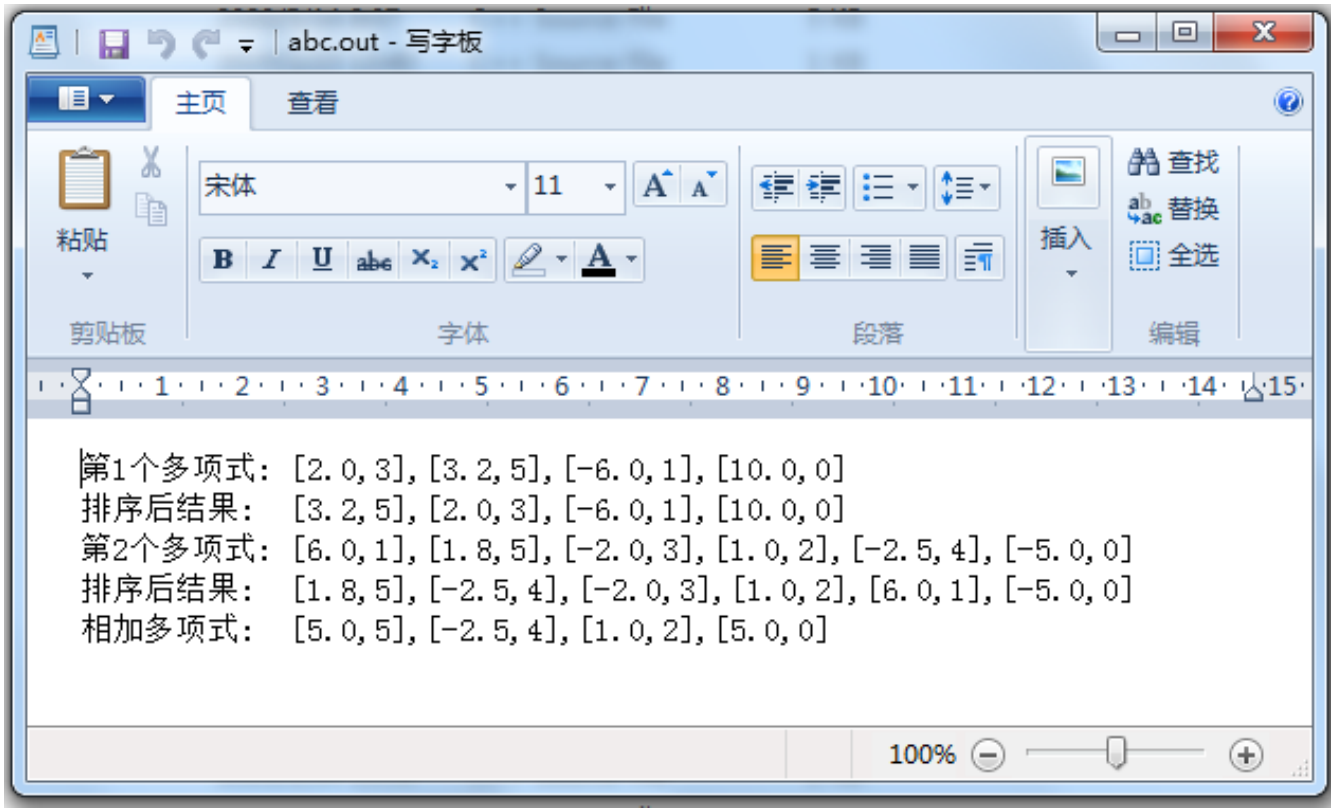
```
    else if (pa->exp<pb->exp)              //归并指数较大的结点pb
    {  s=new PolyNode(pb->coef,pb->exp);//复制产生结点s
       r->next=s; r=s;                     //将结点s链到C末尾
       pb=pb->next;
    }
    else                                   //两结点指数相等的情况
    {  c=pa->coef+pb->coef;                //求两指数相等结点的系数和c
       if (c!=0)                           //系数和不为0的情况
       {  s=new PolyNode(c,pa->exp);       //新建结点s
          r->next=s; r=s;                  //将结点s链到C末尾
       }
       pa=pa->next;
       pb=pb->next;
    }
}
```

```
    if (pb!=NULL) pa=pb;                              //复制余下的结点
    while (pa!=NULL)
    {   s=new PolyNode(pa->coef,pa->exp);     //复制产生结点s
        r->next=s; r=s;                          //将结点s链到C末尾
        pa=pa->next;
    }
    r->next=NULL;                                  //尾结点的next域置为NULL
}
```

4．设计主程序

```
int main()
{  freopen("abc.out","w",stdout);     //输出重定向到abc.out文件
   PolyList A,B,C;                    //建立3个多项式单链表对象
   A.CreateList("abc1.in");
   cout << "第1个多项式: "; A.DispPoly();
   A.Sort();
   cout << "排序后结果:  "; A.DispPoly();
   B.CreateList("abc2.in");
   cout << "第2个多项式: "; B.DispPoly();
   B.Sort();
   cout << "排序后结果:  "; B.DispPoly();
   PolyAdd(A,B,C);
   cout << "相加多项式:  "; C.DispPoly();
   return 0;
}
```

执行程序后打开abc.out文件



2.6 STL中的线性表

- STL中有一类容量称为顺序容器，顺序容器按照线性次序的位置存储数据，即第1个元素，第2个元素，依此类推，简单地说，可以采用这些容量存放线性表。
- STL中的顺序容器有、string、deque和list。这里主要介绍vector和list。



自学，上机编程

C++序列式容器（STL序列式容器）是什么

< 上一节 下一节 >

所谓序列容器，即以线性排列（类似普通数组的存储方式）来存储某一指定类型（例如 int、double 等）的数据，需要特殊说明的是，该类容器并不会自动对存储的元素按照值的大小进行排序。

需要注意的是，序列容器只是一类容器的统称，并不指具体的某个容器，序列容器大致包含以下几类容器：

- array<T,N>（数组容器）：表示可以存储 N 个 T 类型的元素，是 C++ 本身提供的一种容器。此类容器一旦建立，其长度就是固定不变的，这意味着不能增加或删除元素，只能改变某个元素的值；
- vector<T>（向量容器）：用来存放 T 类型的元素，是一个长度可变的序列容器，即在存储空间不足时，会自动申请更多的内存。使用此容器，在尾部增加或删除元素的效率最高（时间复杂度为 O(1) 常数阶），在其它位置插入或删除元素效率较差（时间复杂度为 O(n) 线性阶，其中 n 为容器中元素的个数）；
- deque<T>（双端队列容器）：和 vector 非常相似，区别在于使用该容器不仅尾部插入和删除元素高效，在头部插入或删除元素也同样高效，时间复杂度都是 O(1) 常数阶，但是在容器中某一位置处插入或删除元素，时间复杂度为 O(n) 线性阶；
- list<T>（链表容器）：是一个长度可变的、由 T 类型元素组成的序列，它以双向链表的形式组织元素，在这个序列的任何地方都可以高效地增加或删除元素（时间复杂度都为常数阶 O(1)），但访问容器中任意元素的速度要比前三种容器慢，这是因为 list<T> 必须从第一个元素或最后一个元素开始访问，需要沿着链表移动，直到到达想要的元素。
- forward_list<T>（正向链表容器）：和 list 容器非常类似，只不过它以单链表的形式组织元素，它内部的元素只能从第一个元素开始访问，是一类比链表容器快、更节省内存的容器。

> 注意，其实除此之外，stack<T> 和 queue<T> 本质上也属于序列容器，只不过它们都是在 deque 容器的基础上改头换面而成，通常更习惯称它们为容器适配器，有关它们的介绍，会放到后续章节中。

1. `vector`：动态数组

时间复杂度特性：

- **随机访问**：`O(1)`（连续内存，直接通过下标计算地址）。
- **尾部插入/删除**：均摊`O(1)`（可能触发扩容，但均摊后为常量时间）。
- **中间/头部插入/删除**：`O(n)`（需要移动后续元素）。
- **空间效率**：内存连续，无额外指针开销，但扩容时可能浪费内存。

适用场景：

- **需要高效随机访问**，且插入/删除操作集中在尾部。
- **数据规模变化较小**，或能预分配足够容量避免频繁扩容。

举例：

- **学生成绩管理系统**：存储学生成绩，按索引快速查询或修改成绩，新增成绩通常在末尾追加。

```cpp
vector<int> scores = {85, 90, 78};
scores.push_back(92);          // 尾部插入：O(1)
int third_score = scores[2];  // 随机访问：O(1)
```

2. `list`：双向链表

时间复杂度特性：

- **插入/删除**：`O(1)`（已获得迭代器时，仅需调整指针）。
- **随机访问**：`O(n)`（必须遍历链表）。
- **空间开销**：每个节点需额外存储前后指针（内存碎片化）。

适用场景：

- **频繁在任意位置插入/删除**，且无需随机访问。
- **需要稳定的迭代器**（元素插入/删除不会使其他元素的迭代器失效）。

举例：

- **音乐播放列表**：用户频繁在任意位置添加或删除歌曲，顺序播放时无需随机访问。

```cpp
list<string> playlist = {"Song1", "Song2"};
auto it = playlist.begin();
advance(it, 1);           // 遍历到第二个元素：O(n)
playlist.insert(it, "NewSong"); // 插入：O(1)
```

3. `deque`：双端队列

时间复杂度特性：

- **随机访问**：`O(1)`（通过分段数组索引，略慢于`vector`）。
- **头部/尾部插入/删除**：`O(1)`（无需移动元素）。
- **中间插入/删除**：`O(n)`（类似`vector`，需移动元素）。

适用场景：

- **需要两端高效操作**，同时偶尔随机访问。
- **无法预知数据规模**，且需避免`vector`扩容时的复制开销。

举例：

- **撤销操作历史记录**：用户频繁在两端添加操作（如撤销/重做），偶尔需访问中间某次操作。

```cpp
deque<string> history;
history.push_front("Action1");  // 头部插入：O(1)
history.push_back("Action2");   // 尾部插入：O(1)
string first_action = history[0]; // 随机访问：O(1)
```

总结对比

容器	核心优势	典型场景
`vector`	随机访问、尾部操作高效	动态数组、数据缓存
`list`	任意位置插入/删除高效	链表结构、频繁修改的序列
`deque`	两端操作高效，支持随机访问	队列/栈、需两端扩展的序列

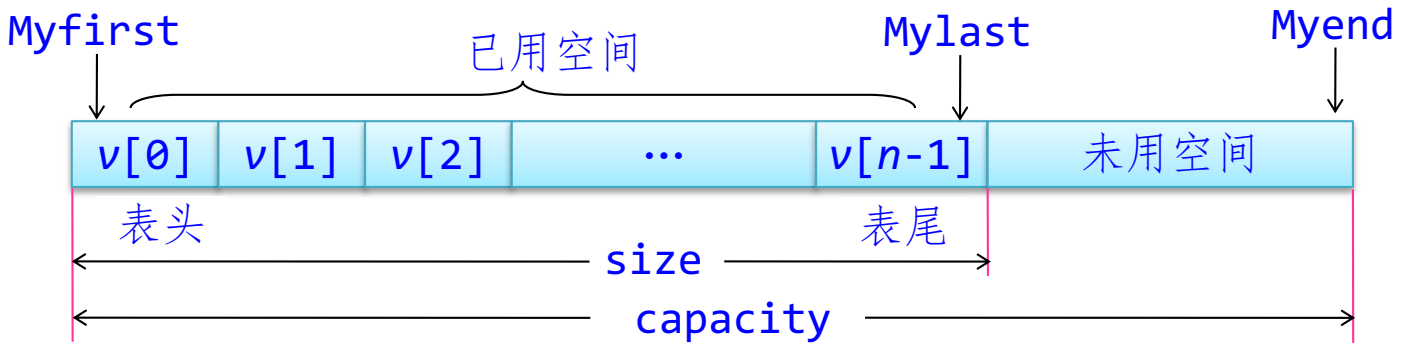
选择建议：

- 优先`vector`：除非中间插入或内存碎片成为瓶颈。
- 优先`list`：需频繁在中间插入/删除且数据规模较大。
- 优先`deque`：需同时支持两端操作和随机访问。

2.6.1 vector向量容器

1. vector向量的基本应用

- vector向量容器是一个可变长的动态数组。
- 根据下标（索引）随机访问某个元素的时间是常数。在尾部添加一个元素的时间大多数情况下也是常数。
- 在中间插入或删除元素时，因为要移动多个元素，因此速度较慢，平均花费的时间和容器中的元素个数成正比。



定义vector容器的几种方式如下：

```
vector<int> v1;                  //定义元素为int的向量v1
vector<int> v2(10);              //指定向量v2的初始大小为10个int元素
vector<double> v3(10, 1.23);     //指定v3的10个初始元素的初值为1.23
vector<int> v4(a, a+5);          //用数组a[0..4]共5个元素初始化v4
```

vector的主要成员函数及其说明

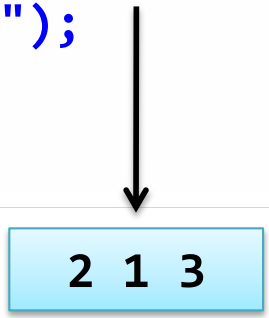
成员函数	说明
empty()	判断当前向量容器是否为空
size()	返回当前向量容器的中的实际元素个数
[]	返回指定下标的元素
reserve(n)	为当前向量容器预分配n个元素的存储空间
capacity()	返回当前向量容器在重新进行内存分配以前所能容纳的元素个数
resize(n)	调整当前向量容器的大小，使其能容纳n个元素
front()	获取当前向量容器的第一个元素
back()	获取当前向量容器的最后一个元素
push_back(e)	在当前向量容器尾部添加了一个元素e
insert(p, e)	在p位置插入元素e，即将元素e插入到迭代器p指定元素之前
pop_back()	删除向量中最后一个元素
erase()	删除当前向量容器中某个迭代器或者迭代器区间指定的元素
clear()	删除当前向量容器中所有元素

迭代器成员函数

成员函数	说明
begin()	返回容器中第一个元素的迭代器
end()	返回容器中尾元素后面一个位置的迭代器
rbegin()	返回容器中尾元素的反向迭代器
rend()	返回容器中首元素前面一个位置的反向迭代器

以下程序说明vector容器的应用

```
#include<iostream>
#include <vector>
using namespace std;
int main()
{  vector<int> myv;              //定义vector容器myv
   vector<int>::iterator it;     //定义myv的正向迭代器it
   myv.push_back(1);             //在myv末尾添加元素1
   it=myv.begin();               //it迭代器指向开头元素1
   myv.insert(it,2);             //在it指向的元素之前插入元素2
   myv.push_back(3);             //在myv末尾添加元素3
   myv.push_back(4);             //在myv末尾添加元素4
   it=myv.end();                 //it迭代器指向尾元素4的后面
   it--;                         //it迭代器指向尾元素4
   myv.erase(it);                //删除元素4
   for (it=myv.begin();it!=myv.end();++it)
      printf("%d ",*it);
   printf("\n");
   return 0;
}
```

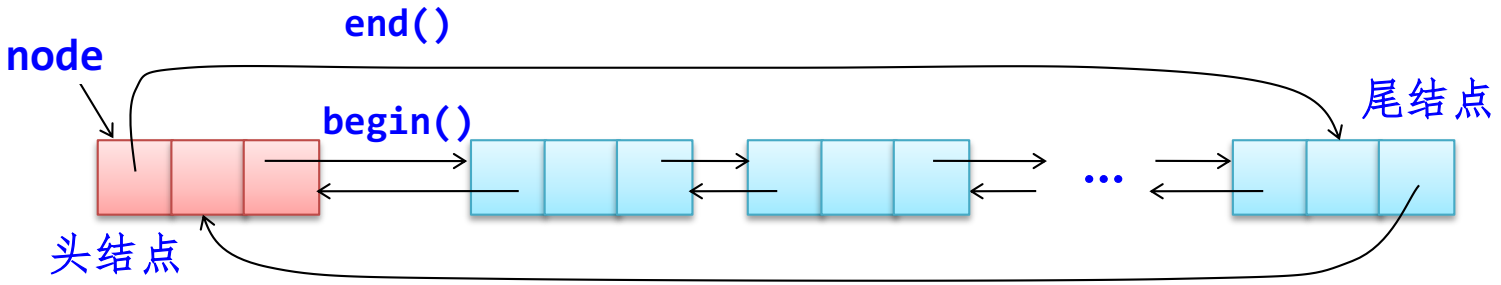


2．vector向量的排序

- STL的算法库提供了丰富的函数，许多函数可以用于vector向量以实现复杂的功能。
- 例如STL的排序算法sort()（用于数组、vector向量等具有随机存取特性的容器）。
- 有关排序算法的原理在第10章介绍，这里仅仅讨论sort()算法的应用。

2.6.2 list链表容器

- list链表容器是一个循环双链表。
- 可以从任何地方快速插入与删除。
- 它的每个结点之间通过指针链接，不能随机访问元素，为了访问表容器中特定的元素，必须从表头开始顺序遍历直到找到匹配的结点。
- list容器插入比vector快，由于对每个结点单独分配空间，所以不存在空间不够而需要重新分配的情况。



定义list容器的几种方式如下：

```
list<int> l1;                    //定义元素为int的链表l1
list<int> l2 (10);               //指定链表l2的初始大小为10个int元素
list<double> l3 (10, 1.23);      //指定l3的10个初始元素的初值为1.23
list<int> l4(a, a+5);            //用数组a[0..4]共5个元素初始化l4
```

list的主要成员函数及其说明

成员函数	说明
empty()	判断链表容器是否为空
size()	返回链表容器中实际元素个数
back()	返回链表容器中尾元素
front()	返回链表容器中头元素
push_back()	在链表尾部插入元素
pop_back()	删除链表容器的尾元素
push_front()	在链表头部插入元素
pop_front()	删除链表容器的头元素
remove()	删除链表容器中所有指定值的元素
remove_if(cmp)	删除链表容器中满足条件的元素
erase()	从链表容器中删除一个或几个元素
unique()	删除链表容器中相邻的重复元素
clear()	删除链表容器中所有的元素
insert(pos, elem)	在pos位置插入元素elem，即将elem插入到迭代器pos指定元素之前
insert(pos,n,elem)	在pos位置插入n个元素elem
insert(pos,pos1,pos2)	在迭代器pos处插入[pos1, pos2)的元素
reverse()	反转链表
sort()	对链表容器中的元素排序

迭代器成员函数

成员函数	说明
begin()	返回容器中第一个元素的迭代器
end()	返回容器中尾元素后面一个位置的迭代器
rbegin()	返回容器中尾元素的反向迭代器
rend()	返回容器中首元素前面一个位置的反向迭代器

```
#include<iostream>
#include <list>
using namespace std;
void disp(list<int>& lst)     //输出lst的所有元素
{  list<int>::iterator it;
   for (it=lst.begin();it!=lst.end();it++)
      printf("%d ",*it);
   printf("\n");
}
```

```
int main()
{  list<int> lst;                    //定义list容器lst
   list<int>::iterator it,start,end;
   lst.push_back(5);                 //添加5个整数5,2,4,1,3
   lst.push_back(2);
   lst.push_back(4);
   lst.push_back(1);
   lst.push_back(3);
   printf("初始lst: "); disp(lst);
   it=lst.begin();                   //it指向首元素5
   start=++lst.begin();                      //start指向第2个元素2
   end=--lst.end();                  //end指向尾元素3
   lst.insert(it,start,end);
   printf("执行lst.insert(it,start,end)\n");
   printf("插入后lst: "); disp(lst);
   return 0;
}
```



```
初始lst: 5 2 4 1 3
执行lst.insert(it,start,end)
插入后lst: 2 4 1 5 2 4 1 3
```